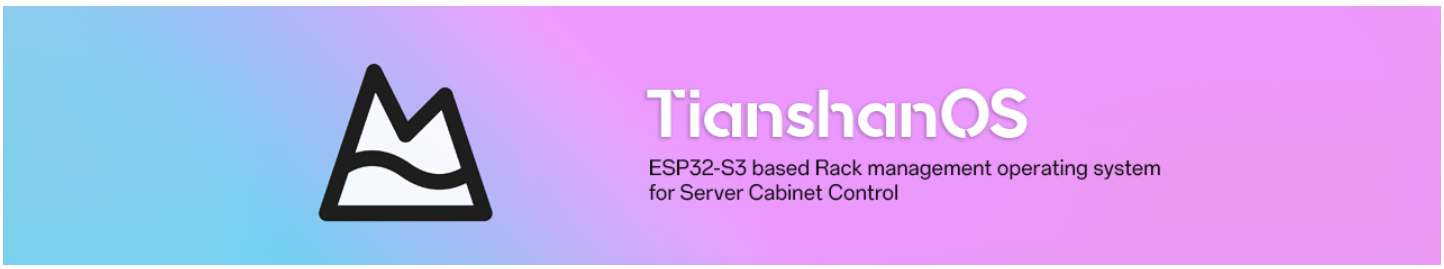
安全页面完整操作指南 / Comprehensive Guide to Security Page



安全页面完整操作指南（admin 用户）

- 概述与准备
- 任务一：生成并使用 SSH 密钥连接远程服务器
- 任务二：管理已部署主机
- 任务三：管理已知主机指纹（防中间人攻击）
- 任务四：管理 HTTPS 证书（mTLS 双向认证）
- 任务五：配置包 (Config Pack)
- 安全存储 vs 文件密钥对比
- 常见问题与故障排查
- 最佳实践建议
- 参考文档链接
- 结语

Comprehensive Guide to Security Page (admin user)

- Overview and Preparation
- Task 1: Generate and use SSH keys to connect to a remote server
- Task 2: Manage deployed hosts
- Task 3: Manage known host fingerprints (to prevent man-in-the-middle attacks)
- Task 4: Manage HTTPS Certificates (mTLS Mutual Authentication)

本文档为 **admin 权限用户**提供 TianshanOS 安全页面的完整操作指南，涵盖 SSH 密钥管理、远程主机连接、HTTPS 证书配置等所有功能的详细使用方法。

安全页面完整操作指南（admin 用户）

概述与准备

安全页面的作用

安全页面是 TianshanOS 的安全管理中心，集中管理以下功能：

1. **SSH 密钥管理**：在设备上生成或导入 SSH 密钥，安全存储在 NVS 加密分区中
2. **远程主机部署**：将公钥部署到远程 Linux 服务器，实现免密登录
3. **已知主机指纹**：记录 SSH 服务器指纹，防止中间人攻击
4. **HTTPS 证书管理**：配置设备的 mTLS 证书，用于双向身份验证

admin 用户的权限范围

作为 **admin 用户**，您可以使用安全页面的**所有功能**，包括：

- 生成、删除、导出 SSH 密钥
- 部署公钥到远程服务器
- 撤销已部署的公钥
- 管理已知主机指纹
- 生成和管理 HTTPS 证书
- 管理配置包（导入、导出、查看）

注意：以下页面需要 **root 权限**，admin 用户无法访问：

- 终端页面
- 自动化页面
- 指令页面

如何进入安全页面

1. 在浏览器中访问设备的 WebUI（如 `http://192.168.4.1`）
2. 使用 admin 账户登录（默认密码： `rm01` ，**强烈建议首次登录后立即修改**）
3. 在顶部导航栏点击 **「安全」** 链接
4. 进入安全页面（URL 路由为 `#/security`）

页面结构概览

安全页面从上到下分为五个区块：

区块	功能概述
密钥管理	生成、查看、导出、部署、撤销、删除 SSH 密钥
已部署主机	管理已部署公钥的远程服务器列表；支持导入主机配置；测试连接、撤销公钥、移除记录
已知主机指纹	查看和删除 SSH 服务器指纹，防止中间人攻击
HTTPS 证书	生成设备证书、CSR、安装 CA 签发的证书，配置 mTLS
配置包 (Config Pack)	导出设备证书、导入/导出加密配置包、查看配置包列表，用于设备间安全配置分发

任务一：生成并使用 SSH 密钥连接远程服务器

本任务演示如何从零开始生成 SSH 密钥，部署到远程 Linux 服务器（如 Jetson AGX Orin、工作站等），并实现免密登录。

2.1 生成 SSH 密钥

操作步骤

1. **进入密钥管理区域**：在安全页面顶部找到「密钥管理」区块
2. **点击生成按钮**：点击 **「生成新密钥」** 按钮
3. **填写生成表单**：在弹出的「生成新密钥」弹窗中填写以下信息：

表单字段详解

字段	必填	说明	填写示例
密钥 ID	是	密钥的唯一标识符，用于后续引用。命名规则：字母、数字、下划线，不可重复	`default`、`agx_key`、`lpmu_prod`
密钥类型	是	选择加密算法和密钥长度	见下方「密钥类型选择」
备注	否	密钥的用途说明，便于日后识别	`用于连接 AGX Orin`、`测试环境密钥`
别名	否	替代密钥 ID 显示的友好名称，启用「隐藏密钥」时建议填写	`AGX 生产环境`、`主服务器密钥`
允许导出私钥	否		仅在需要备份时勾选

		勾选后可导出私钥进行备份或迁移。**安全风险**：私钥泄露可导致未授权访问	
隐藏密钥 ID	否	勾选后低权限用户无法看到真实密钥 ID，仅显示别名或掩码，适用于多用户环境	多人共用设备时勾选

密钥类型选择

类型	密钥长度	生成时间	兼容性	推荐场景
RSA 2048-bit	2048 位	5-10 秒	兼容性最好	**推荐**：日常使用
RSA 4096-bit	4096 位	30-60 秒	兼容性最好	高安全场景，生成较慢
ECDSA P-256	256 位	3-5 秒	一般	**注意**：当前不支持 SSH 公钥认证
ECDSA P-384	384 位	3-5 秒	一般	**注意**：当前不支持 SSH 公钥认证

重要提示：当前版本 ECDSA 密钥暂不支持 SSH 公钥认证，请选择 **RSA 2048-bit** 或 **RSA 4096-bit**。

生成流程

4. **提交表单**：填写完成后点击 **「生成」** 按钮
5. **等待生成**：
   - RSA 2048：约 5-10 秒
   - RSA 4096：约 30-60 秒（请耐心等待，不要关闭页面）
   - 弹窗会显示生成进度
6. **确认成功**：

    - 弹窗显示「密钥生成成功」
    - 关闭弹窗后，新密钥出现在「密钥管理」表格中
7. **查看密钥信息**：表格显示 ID、类型、备注、创建时间、是否可导出

注意事项

- **密钥 ID 唯一性**：如果输入的 ID 已存在，会提示错误，需更换其他 ID
- **不可中断**：生成过程中不要刷新页面或关闭浏览器
- **永久存储**：密钥生成后安全存储在设备的 NVS（非易失性存储）中，重启不丢失
- **存储限制**：NVS 最多可存储约 8 个 RSA-4096 密钥或 32 个 ECDSA 密钥

2.2 部署公钥到远程服务器

生成密钥后，需要将**公钥**部署到远程 Linux 服务器，才能实现免密登录。

前提条件

- 已在「密钥管理」中生成至少一个密钥
- 知道远程服务器的 IP 地址或主机名
- 知道远程服务器的用户名和密码（用于首次部署）
- 设备与远程服务器网络连通

操作步骤

1. **选择密钥**：在「密钥管理」表格中找到要部署的密钥行
2. **点击部署按钮**：点击该密钥行最右侧的 **「部署」** 按钮（蓝色）
3. **填写部署表单**：在弹出的「部署公钥到远程服务器」弹窗中填写：

部署表单字段

字段	必填	说明	填写示例
目标主机	是	远程服务器的 IP 地址或主机名	`192.168.55.100`、`agx.local`、`10.10.99.98`
用户名	是	远程服务器的登录用户名	`root`、`nvidia`、`ubuntu`
端口	否	SSH 服务端口，默认 22。如果服务器使用非标准端口需修改	`22`（默认）、`2222`、`22022`
认证密码	是	首次部署时用于身份验证的密码。部署成功后，后续连接无需密码	您的服务器登录密码

4. **开始部署**：点击 **「开始部署」** 按钮
5. **查看进度**：弹窗中会显示部署过程输出，包括：
   - 连接服务器
   - 验证主机密钥（首次连接会提示信任指纹，详见「任务三」）
   - 上传公钥到 `~/.ssh/authorized_keys`
   - 验证部署结果
6. **确认成功**：
   - 显示「公钥部署成功」
   - 显示「连接验证成功」
   - 该主机自动添加到「已部署主机」列表

部署原理

部署过程会执行以下操作：

1. 使用提供的密码通过 SSH 连接到远程服务器
2. 读取密钥的公钥内容（OpenSSH 格式）
3. 将公钥追加到远程服务器的 `~/.ssh/authorized_keys` 文件末尾
4. 验证部署是否成功（尝试使用密钥连接）

技术细节：类似 Linux 的 `ssh-copy-id` 命令。

常见错误处理

错误提示	可能原因	解决方法
连接超时	网络不通、防火墙阻止、服务器未开启 SSH	检查网络连通性（ping）、确认 SSH 服务运行（`systemctl status sshd`）、检查防火墙规则
密码错误	密码输入错误、用户名不正确	重新确认用户名和密码，注意大小写
权限被拒绝	SSH 配置不允许密码登录	在服务器 `/etc/ssh/sshd_config` 中启用 `PasswordAuthentication yes`
主机指纹验证失败	首次连接需要信任主机	详见「任务三：管理已知主机指纹」章节
authorized_keys 权限错误	远程 `~/.ssh` 目录权限不正确	确保 `~/.ssh` 权限为 700，`authorized_keys` 权限为 600

2.3 使用密钥免密登录

部署成功后，您可以使用密钥连接远程服务器，无需再输入密码。

在 WebUI 中测试连接

1. **进入已部署主机区域**：滚动到「已部署主机」表格
2. **找到目标主机**：在列表中找到刚才部署的主机
3. **点击测试按钮**：点击该主机行的 **「测试」** 按钮
4. **查看结果**：
   - **成功**：显示「连接成功」和远程用户名
   - **失败**：显示错误信息（网络问题、密钥不匹配、服务器配置等）

在终端中使用密钥连接

如果您有 **root 权限**可以访问终端页面，或者通过其他方式访问设备的 CLI（如串口），可以使用以下命令：

代码块

```
# 启动交互式 Shell
ssh --host 192.168.1.100 --user nvidia --keyid agx_key --shell

# 执行单条命令
ssh --host 192.168.1.100 --user nvidia --keyid agx_key --exec "nvidia-smi"

# 仅测试连接
ssh --test --host 192.168.1.100 --user nvidia --keyid agx_key
```

参数说明：

- `--host`：远程服务器地址
- `--user`：登录用户名
- `--keyid`：使用安全存储中的密钥 ID（与部署时的密钥 ID 一致）
- `--shell`：启动交互式 Shell

- `--exec`：执行单条命令后退出

连接失败排查

如果免密登录失败，按以下顺序排查：

1. **检查密钥是否正确部署**：
   - 在「已部署主机」列表中查看该主机是否存在
   - 确认使用的密钥 ID 与部署时一致

2. **检查网络连通性**：
   - 确保设备与服务器网络可达
   - 尝试 ping 服务器 IP

3. **检查服务器配置**：
   - 确认 SSH 服务正在运行
   - 检查 `/etc/ssh/sshd_config` 中 `PubkeyAuthentication yes`
   - 确认 `~/.ssh/authorized_keys` 文件存在且包含公钥

4. **查看详细日志**：在终端中使用 `--verbose` 参数查看详细连接日志

2.4 撤销已部署的公钥

当不再需要访问某服务器，或者怀疑密钥泄露时，应及时撤销已部署的公钥。

何时需要撤销

- 不再需要访问该服务器

- 密钥可能泄露，需要更换
- 服务器用户变更，需要清理旧密钥
- 定期安全审查，清理过期访问权限

撤销方式一：从密钥管理撤销

适用场景：知道具体要撤销哪个密钥、哪个服务器

1. **选择密钥**：在「密钥管理」表格中找到要撤销的密钥
2. **点击撤销按钮**：点击 **「撤销」** 按钮（橙色）
3. **填写撤销表单**：

字段	必填	说明
目标主机	是	要撤销公钥的服务器地址
用户名	是	服务器登录用户名
端口	否	SSH 端口，默认 22
认证密码	是	服务器密码（因为密钥可能已失效，需要密码认证）

4. **确认撤销**：点击 **「撤销公钥」** 按钮
5. **查看结果**：
    - 显示从 `authorized_keys` 中删除了几行公钥
    - 本地「已部署主机」记录自动删除

撤销方式二：从已部署主机撤销

适用场景：在「已部署主机」列表中快速撤销

1. **进入已部署主机区域**：滚动到「已部署主机」表格
2. **点击撤销按钮**：在目标主机行点击 **「撤销」** 按钮（红色）
3. **输入密码**：在弹出的撤销确认弹窗中输入服务器密码
4. **确认撤销**：点击 **「撤销并移除」** 按钮
5. **等待完成**：系统会：
    - 连接到远程服务器
    - 从 `authorized_keys` 删除对应公钥
    - 删除本地「已部署主机」记录

仅移除本地记录（不撤销远程公钥）

适用场景：

- 服务器已停用或无法访问
- 公钥已在服务器端手动删除
- 仅需清理本地记录，保留远程公钥

操作步骤：

1. 在「已部署主机」表格中找到目标主机
2. 点击 **「移除」** 按钮（灰色）
3. 确认移除
4. 本地记录删除，远程服务器的 `authorized_keys` **不会**被修改

撤销 vs 移除的区别

操作	远程公钥	本地记录	需要密码	适用场景
撤销	删除	删除	需要	彻底取消访问权限
移除	保留	删除	不需要	清理本地记录，保留远程访问

2.5 导出密钥（备份或迁移）

导出公钥

用途：

- 手动添加到其他服务器的 `authorized_keys`
- 分享给其他用户或设备
- 备份公钥内容

操作步骤：

1. 在「密钥管理」表格中找到目标密钥
2. 点击 **「公钥」** 按钮
3. 在弹出的「公钥导出」弹窗中：
    - 查看 OpenSSH 格式的公钥内容（如 `ssh-rsa AAAAB3NzaC1yc2E...`）
    - 点击 **「复制到剪贴板」** 复制内容
    - 或点击 **「下载文件」** 保存为 `.pub` 文件

4. **使用公钥：**
    - 粘贴到远程服务器的 `~/.ssh/authorized_keys` 文件
    - 或通过邮件、聊天工具发送给其他管理员

导出私钥

重要安全警告：私钥是高度敏感的安全凭证，泄露后任何人都可以使用它访问已部署的服务器。仅在以下场景导出私钥：

- 密钥备份（存储在安全的离线介质）
- 密钥迁移到其他设备

- 在其他 SSH 客户端（如桌面电脑）使用该密钥

前提条件：

- 密钥生成时勾选了 **「允许导出私钥」**
- 如果未勾选，该密钥的「私钥」按钮会显示为灰色不可用

操作步骤：

1. 在「密钥管理」表格中找到可导出的密钥
2. 点击 **「私钥」** 按钮
3. **阅读安全警告**：系统会弹出警告对话框，提示私钥安全风险
4. **确认继续**：点击「确定」
5. 在「私钥导出」弹窗中：
   - 查看 PEM 格式的私钥内容（`-----BEGIN RSA PRIVATE KEY-----` ...）
   - 点击 **「复制到剪贴板」** 复制内容
   - 或点击 **「下载文件」** 保存为 `.pem` 文件

私钥安全处理建议

导出私钥后，务必遵循以下安全实践：

1. **安全传输**：
   - 不要通过电子邮件发送私钥
   - 不要在聊天软件中粘贴私钥
   - 不要上传到云存储（除非加密）
   - 使用加密的 U 盘、移动硬盘
   - 本地加密后再传输

2. **安全存储**：
   - 存储在加密的磁盘或保险柜
   - 设置文件权限为 600（仅所有者可读写）

- 使用密码管理器存储（如 1Password、Bitwarden）

3. **使用后处理**：
   - 临时使用后立即删除文件
   - 清空剪贴板历史
   - 不要在公共电脑上使用私钥

4. **定期审查**：
   - 检查是否有未授权的公钥使用
   - 怀疑泄露时立即撤销并重新生成

2.6 删除不再需要的密钥

删除前的检查清单

在删除密钥前，请确认：

- [ ] 该密钥没有部署到重要服务器，或已完成撤销
- [ ] 已导出私钥备份（如果需要）
- [ ] 了解删除后无法恢复（除非有备份）
- [ ] 检查「已部署主机」列表，确认没有使用该密钥的主机

操作步骤

1. **选择密钥**：在「密钥管理」表格中找到要删除的密钥
2. **点击删除按钮**：点击 **「删除」** 按钮（红色）
3. **确认删除**：系统会弹出确认对话框，提示删除后无法恢复
4. **最终确认**：点击「确定」执行删除

5. **验证结果**：密钥从列表中消失

删除后果

- 密钥从 NVS 永久删除，释放存储空间
- 已部署主机记录**仍然保留**（需手动撤销或移除）
- 无法使用该密钥连接已部署的服务器（除非重新生成并部署）
- 无法恢复（除非之前导出了私钥备份）

删除密钥后的清理工作

1. **清理已部署主机**：
    - 进入「已部署主机」列表
    - 找到使用该密钥的主机
    - 点击「移除」删除本地记录
    - （可选）如果服务器仍可访问，先撤销公钥再删除密钥

2. **通知相关人员**（多用户环境）：
    - 告知其他管理员该密钥已删除
    - 更新访问文档和密钥清单

任务二：管理已部署主机

「已部署主机」区块记录了哪些远程服务器已部署了设备的公钥，方便您统一管理和快速测试连接。

3.0 导入主机配置（可选）

若您通过终端 CLI（如 `ssh --copyid`）或手动方式将公钥部署到了远程服务器，这些主机不会自动出现在「已部署主机」列表中。您可以通过**导入主机配置**，将来自其他设备或备份的 SSH 主机配置（.tscfg 配置包）导入到本设备，使列表与实际情况一致，便于在 WebUI 中统一测试、撤销或移除。

操作步骤：

1. **进入已部署主机区域**：在安全页面找到「已部署主机」区块，右侧有 **「导入主机」** 按钮
2. **点击导入主机**：点击 **「导入主机」** 按钮，打开「导入 SSH 主机配置」弹窗
3. **选择文件**：选择本地的 `.tscfg` 配置包文件（通常由其他设备的「导出主机配置」或配置包系统生成）
4. **预览与确认**：系统会验证配置包签名并显示配置 ID、签名者、备注；若该配置已存在会提示是否覆盖
5. **可选**：勾选「覆盖已存在的配置」以覆盖同 ID 的现有配置
6. **确认导入**：点击 **「确认导入」**，导入成功后刷新页面即可在「已部署主机」列表中看到对应记录

说明：导入的配置包需为有效的、经签名的 SSH 主机配置格式；仅用于同步「已部署主机」列表，不会修改远程服务器上的 `authorized_keys`。

3.0.1 导出主机配置（单条）

在「已部署主机」表格中，每一行主机都有 **「导出」** 按钮，可将该主机的配置导出为加密的 `.tscfg` 文件，用于备份或导入到另一台设备。

操作步骤：

1. **选择主机**：在「已部署主机」表格中找到要导出的主机
2. **点击导出**：点击该行的 **「导出」** 按钮（蓝色），打开「导出 SSH 主机配置」弹窗
3. **目标设备证书（可选）**：
   - **留空**：使用本机证书加密（自加密），导出的 .tscfg 只能在本机或拥有本机证书的设备上解密，适合备份
   - **粘贴目标设备证书 PEM**：若要将该主机配置发给另一台设备，可粘贴对方在「配置包」区块导出的设备证书，导出的 .tscfg 仅该设备能解密并导入

4. **确认导出**：点击 **「导出」**，系统生成 .tscfg 并提示下载或保存路径

典型用途：备份单台主机配置、在多台 TianshanOS 设备间同步同一主机配置、与「导入主机」配合完成设备迁移。

3.1 查看已部署主机列表

列表字段说明

字段	说明	示例
主机 ID	自动生成的唯一标识，格式：`密钥ID@主机地址`	`agx_key@192.168.55.100`、`default@agx.local`
地址	远程服务器的 IP 地址或主机名	`192.168.55.100`、`10.10.99.98`
端口	SSH 服务端口	`22`、`2222`
用户名	远程登录用户	`nvidia`、`root`、`ubuntu`
部署密钥	使用的密钥 ID（显示为徽章）	`agx_key`、`default`
操作	每行可用按钮	测试、导出、撤销、移除（见下文各小节）

列表何时更新

- 通过 WebUI「部署」功能成功部署后自动添加
- 通过 WebUI「撤销」功能撤销后自动删除
- 手动点击「移除」后删除
- **注意**：通过终端 CLI 或手动添加到 `authorized_keys` 的主机**不会**自动出现在此列表；可使用「导入主机」导入 .tscfg 配置包以同步列表

空列表处理

如果列表显示「暂无已部署主机」：

- 说明您还没有通过 WebUI 部署过任何公钥
- 请按「任务一 2.2」的步骤部署公钥
- 部署成功后主机会自动出现在此列表

与指令页面的关系

指令页面（需 root 权限）中「选择主机」下拉列表的数据来源即为本安全页的「已部署主机」。在指令页导出指令配置时，可选择**同时导出依赖的主机配置**，便于在另一台设备上导入指令时一并导入所需主机，无需单独再在安全页导入主机。

3.2 测试 SSH 连接

快速验证某个已部署主机的连接状态。

操作步骤

1. **选择主机**：在「已部署主机」表格中找到要测试的主机
2. **点击测试按钮**：点击 **「测试」** 按钮
3. **等待测试**：系统会执行以下操作：
   - 使用部署的密钥连接服务器
   - 执行简单命令（如 `whoami`）验证连接
   - 检查返回结果
4. **查看结果**：在弹窗或结果区域显示测试结果

测试结果解读

成功示例：

```
SSH 连接测试成功
主机：192.168.55.100:22
用户：nvidia
命令：whoami
输出：nvidia
```

失败示例：

```
SSH 连接测试失败
错误：连接超时（Connection timeout）
建议：检查网络连通性和防火墙设置
```

失败处理建议

错误类型	处理方法
连接超时	检查网络、确认服务器在线、检查防火墙
密钥认证失败	公钥可能被删除，重新部署或检查 `authorized_keys`
主机指纹不匹配	服务器可能重装，更新主机指纹（见「任务三 4.4」）
权限被拒绝	检查服务器 SSH 配置和公钥文件权限

3.3 撤销公钥

详见「任务一 2.4 撤销方式二：从已部署主机撤销」。

3.4 移除主机记录

与撤销的区别

操作	本地记录	远程公钥	适用场景
撤销	删除	删除	需要彻底取消访问权限
移除	删除	保留	服务器已停用、公钥已手动删除、仅清理本地

适用场景

- 服务器已停用或销毁
- 公钥已在服务器端手动删除
- 主机记录错误，需要清理
- 仅需清理本地列表，远程公钥保留

操作步骤

1. **选择主机**：在「已部署主机」表格中找到目标主机
2. **点击移除按钮**：点击 **「移除」** 按钮（灰色）

3. **确认移除**：系统弹出确认对话框
4. **最终确认**：点击「确定」
5. **验证结果**：主机从列表中消失

注意：移除操作**不会**连接远程服务器，仅删除本地数据库中的记录。如果需要删除远程公钥，请使用「撤销」功能。

任务三：管理已知主机指纹（防中间人攻击）

「已知主机指纹」功能通过记录 SSH 服务器的主机密钥指纹，防止连接到伪造的服务器（中间人攻击）。

4.1 什么是已知主机指纹

TOFU（Trust On First Use）机制

工作原理：

1. **首次连接**某个 SSH 服务器时，服务器会发送其主机密钥（公钥）
2. 客户端计算该密钥的 **SHA256 哈希值**（指纹）
3. 提示用户是否信任该指纹
4. 用户确认后，指纹保存到设备的 NVS 中
5. **后续连接**时，自动对比服务器指纹与保存的指纹
6. 如果匹配，静默通过；如果不匹配，发出**安全警告**

指纹的作用

- **防止中间人攻击**：确保连接到的是真实服务器，而非攻击者伪造的服务器
- **服务器身份验证**：验证服务器的密钥没有被篡改

- **连接记录**：追踪曾经连接过哪些服务器

指纹格式

指纹是服务器主机密钥的 SHA256 哈希值，格式如下：

代码块

```
SHA256:xYzAbC123456789aBcDeF/gHiJkLmNoPqRsTuVwXyZ01=
```

4.2 查看已知主机指纹列表

列表字段说明

字段	说明	示例
主机	SSH 服务器的地址	`192.168.1.100`、`agx.local`
端口	SSH 服务端口	`22`、`2222`
密钥类型	服务器使用的主机密钥算法	`ssh-rsa`、`ssh-ed25519`、`ecdsa-sha2-nistp256`
指纹（SHA256）	服务器主机密钥的哈希值	`SHA256:xYzAbC...`（部分显示）
添加时间	首次信任该服务器的时间	`2026-01-30 14:30:00`

查看完整指纹

列表中指纹可能被截断显示，查看完整指纹：

1. **点击查看按钮**：在目标主机行点击 **「查看」** 按钮
2. **查看详情**：系统在弹窗中显示该主机的**完整 SHA256 指纹**及主机、端口、密钥类型，便于与服务器端 `ssh-keygen -lf` 的输出逐字对比
3. **对比验证**（可选）：在高安全场景下，将弹窗中的指纹与服务器官方指纹对比，确认一致后再信任

获取服务器端指纹（在服务器上运行）：

代码块

```
# ECDSA 密钥
ssh-keygen -lf /etc/ssh/ssh_host_ecdsa_key.pub

# ED25519 密钥
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

# RSA 密钥
ssh-keygen -lf /etc/ssh/ssh_host_rsa_key.pub
```

输出示例：

代码块

```
256 SHA256:xYzAbC123456789aBcDeF/gHiJkLmNoPqRsTuVwXyZ01= root@server (ECDSA)
```

4.3 首次连接新服务器

连接流程

当您首次连接一个新的 SSH 服务器（部署公钥或使用密钥连接）时，系统会提示信任主机：

步骤 1：发起连接

- 执行部署公钥或 SSH 连接操作

步骤 2：系统提示

代码块

```
┌──────────────────────────────────────────────────────────────────────┐
│  NEW HOST - First time connecting to this server                     │
└──────────────────────────────────────────────────────────────────────┘

Host: 192.168.1.100:22
Key Type: ecdsa-sha2-nistp256
Fingerprint: SHA256:xYzAbC123456789aBcDeF/gHiJkLmNoPqRsTuVwXyZ01=

This is the first time connecting to this host.
Trust this host and continue? (yes/no):
```

步骤 3：用户决策

您有两个选择：

输入	结果	适用场景
yes	信任该服务器，指纹保存到 NVS，继续连接	确认是目标服务器
no	中止连接，不保存指纹	不确定服务器身份，或输入了错误的 IP

步骤 4：信任后

- 指纹保存到设备的「已知主机指纹」列表
- 本次连接继续执行
- 后续连接该服务器时自动验证，无需再次确认

如何验证指纹真实性（高安全场景）

在企业生产环境或高安全要求的场景下，首次连接前应验证指纹：

1. **获取官方指纹**：
   - 联系服务器管理员，要求提供官方主机密钥指纹
   - 或者直接登录服务器运行 `ssh-keygen -lf /etc/ssh/ssh_host_*_key.pub`

2. **对比指纹**：
   - 将系统显示的指纹与官方指纹逐字符对比
   - 确保完全一致（包括密钥类型）

3. **确认后信任**：
   - 指纹一致：输入 `yes` 信任
   - 指纹不一致：输入 `no` 中止，排查问题

4.4 主机指纹不匹配警告

触发场景

当您连接到一个已保存指纹的服务器时，如果服务器返回的指纹与保存的不一致，系统会显示**安全警告**。

可能原因：

原因	风险等级	说明

中间人攻击（MITM）	高危	攻击者在网络中拦截并伪造服务器
服务器重装系统	合法	重装 Linux 后 SSH 主机密钥重新生成
服务器更换密钥	合法	管理员手动重新生成了 SSH 主机密钥
IP 地址重新分配	合法	该 IP 现在分配给了不同的服务器

警告弹窗内容

系统会弹出红色警告弹窗：

代码块

```
安全警告：主机指纹不匹配

主机密钥已更改！这可能表明：
• 中间人攻击（Man-in-the-Middle Attack）
• 服务器重新安装或密钥重新生成
• IP 地址被分配给了不同的服务器

主机：192.168.1.100:22
存储的指纹：SHA256:OldFingerprint123...
当前指纹：SHA256:NewFingerprint456...

建议：如果您确认服务器已重装或密钥已更新，可以点击"更新主机密钥"
移除旧记录，然后重新连接以信任新密钥。
```

正确处理步骤

步骤 1：停止连接

- 不要盲目点击「更新主机密钥」
- 先核实情况

步骤 2：核实变更

选择以下方式之一核实：

1. **联系服务器管理员**：
   - 询问是否最近重装了系统
   - 询问是否重新生成了 SSH 主机密钥
   - 要求提供新的官方指纹

2. **检查服务器日志**（如果可访问）：
   - 查看系统安装日期：`ls -l /var/log/installer/`
   - 查看 SSH 密钥生成时间：`ls -l /etc/ssh/ssh_host_*_key`

3. **物理验证**（如果可接触服务器）：
   - 直接在服务器上运行 `ssh-keygen -lf /etc/ssh/ssh_host_*_key.pub`
   - 对比弹窗中的「当前指纹」

步骤 3：根据核实结果处理

情况 A：确认是合法变更（服务器重装、密钥更换）

1. 在警告弹窗中点击 **「更新主机密钥」** 按钮
2. 旧指纹被删除
3. 重新执行连接操作
4. 系统会再次提示信任新指纹（按 4.3 首次连接流程操作）

情况 B：无法确认或怀疑攻击

1. 点击 **「取消」** 按钮
2. **不要**继续连接该服务器
3. 立即排查：
   - 检查网络环境（是否在不安全的公共 WiFi）

- 联系网络管理员检查是否有 ARP 欺骗、DNS 劫持
- 使用其他设备或网络尝试连接，对比指纹

错误操作的后果

- **盲目信任新指纹**：如果确实是中间人攻击，您的密码和数据会被攻击者窃取
- **忽略警告**：多次忽略后可能形成习惯，降低安全防护意识

安全原则：宁可中断连接排查，也不要冒险信任未验证的指纹。

4.5 删除主机指纹

适用场景

- 服务器已永久停用或销毁
- 服务器已重装，需要清理旧指纹并重新信任
- IP 地址已分配给其他服务器，旧记录无效
- 定期清理不再使用的服务器记录

操作步骤

1. **选择主机**：在「已知主机指纹」表格中找到要删除的主机
2. **点击删除按钮**：点击 **「删除」** 按钮
3. **确认删除**：系统弹出确认对话框
4. **最终确认**：点击「确定」
5. **验证结果**：主机从列表中消失

删除后果

- 指纹记录从 NVS 永久删除
- 下次连接该服务器时，会再次提示信任指纹（按首次连接流程）
- 如果服务器重装，**必须先删除旧指纹**，否则会持续提示指纹不匹配

批量删除

如果需要清空所有已知主机指纹（如测试环境重置），可以通过终端 CLI 执行：

代码块

```
hosts --clear --yes
```

警告：此操作会删除所有已知主机指纹，谨慎使用。

4.6 已知主机列表为空的处理

可能原因

如果打开安全页面时，「已知主机指纹」区域始终显示「暂无已知主机」，可能是：

1. **首次使用**：您还没有连接过任何 SSH 服务器
2. **模块未初始化**：安全服务启动时已知主机模块未正确初始化（已知 Bug）

解决方法

方法 1：触发初始化

- 执行一次 SSH 连接操作（如部署公钥）
- 信任主机后，刷新页面
- 列表应该会显示记录

方法 2：检查初始化

- 如果方法 1 无效，参考项目《故障排除》文档中「已知主机列表初始为空」章节
- 可能需要检查安全服务初始化是否调用了 `ts_known_hosts_init()`

任务四：管理 HTTPS 证书（mTLS 双向认证）

「HTTPS 证书」区块用于配置设备的 mTLS（双向 TLS 认证）证书，使设备可以作为 HTTPS 服务器，并与客户端进行双向身份验证。

5.1 HTTPS 证书概述

什么是 mTLS

mTLS（Mutual TLS） 是双向 TLS 认证：

- **传统 HTTPS**：客户端验证服务器证书（单向）
- **mTLS**：服务器和客户端互相验证证书（双向）

优势：

- 双向身份认证，防止未授权客户端访问
- 防止中间人攻击
- 基于证书的访问控制（无需密码）

证书体系结构

代码块

```
┌──────────────────┐
│    Root CA    │  信任锚点（离线保存）
│   20年有效期   │
└──────────────────┘
          │ 签发
          ▼
┌──────────────────┐
│Intermediate │  在线签发证书
│      CA       │   （如 step-ca 服务器）
│   10年有效期   │
└──────────────────┘
          │ 签发
          ▼
┌──────────────────┐
│ 设备证书      │  本设备的身份证明
│（本设备）      │   （存储在 NVS）
│   1年有效期   │
└──────────────────┘
```

私钥不离开设备原则

安全设计：

- 设备私钥在本地生成，永不导出
- 仅导出 CSR（证书签名请求，不含私钥）
- CA 使用 CSR 签发证书，CA 无法获得设备私钥
- 私钥始终存储在设备的 NVS 中，即使固件升级也保留

对比 SSH 密钥：

- SSH 密钥可选择「允许导出私钥」
- HTTPS 证书私钥**永不可导出**（更严格）

5.2 查看证书状态

状态卡片信息

在「HTTPS 证书」区块顶部，有一个状态卡片实时显示证书状态。

状态与含义

状态文本	说明
加载中...	正在获取证书状态
未生成密钥	尚未生成密钥对，需要先生成
已生成密钥，未安装证书	密钥对已生成，但未安装 CA 签发的证书
已激活	证书已安装且有效
即将过期	证书在 30 天内过期，建议续期
已过期	证书已过期，mTLS 功能不可用

证书详情（已安装时）

当证书状态为「已激活」或「即将过期」时，卡片会展开显示详细信息：

字段	说明	示例
主体 CN	设备的唯一标识（Common Name）	`TIANSHAN-RM01-0001`

签发者	签发该证书的 CA 名称	`TianShanOS Intermediate CA`
生效时间	证书的起始有效时间	`2026-01-27 00:00:00`
过期时间	证书的结束有效时间	`2027-01-27 00:00:00`
序列号	证书的唯一序列号（16 进制）	`01:23:45:67:89:AB`
有效状态	当前是否在有效期内	有效 / 已过期
剩余天数	距离过期还有多少天	`365 天`、`28 天`（<30 天显示警告）

状态卡片显示规则：

- **剩余天数**：仅在证书已安装且处于有效期内时显示；未安装证书或已过期时不显示此项。
- **「即将过期」角标/状态**：当证书有效但距离过期不足 30 天时，状态文本会变为「即将过期」，并可能显示角标提醒，建议尽快续期。
- 卡片展开的详细信息（主体 CN、签发者、生效/过期时间、序列号、有效状态、剩余天数）仅在状态为「已激活」或「即将过期」时显示。

按钮状态说明

HTTPS 证书区块下方有 6 个操作按钮，根据当前状态启用或禁用：

按钮	未生成密钥	已生成密钥	已安装证书	说明
生成密钥对	可用	可用	可用	任何时候都可以生成（会覆盖旧密钥）
生成 CSR	禁用	可用	可用	需要先有密钥对
安装证书	禁用	可用	可用	需要先有密钥对
安装 CA	禁用	可用	可用	需要先有密钥对
查看证书	禁用	禁用	可用	需要已安装证书

删除凭证	禁用	可用	可用	有密钥或证书时可删除

5.3 完整证书申请流程

从零开始配置设备的 mTLS 证书，完整流程包括 5 个步骤。下文中提到的按钮与弹窗名称（如「生成密钥对」「证书签名请求 (CSR)」「安装设备证书」「安装 CA 证书链」「查看设备证书」）均与安全页 WebUI 一致，便于在界面上对应操作。

流程总览

代码块

```
步骤 1: 生成密钥对（设备端）
    ↓
步骤 2: 生成 CSR（设备端）
    ↓
步骤 3: 提交 CA 签发（外部 CA 服务器）
    ↓
步骤 4: 安装设备证书（设备端）
    ↓
步骤 5: 安装 CA 证书链（设备端）
```

5.3.1 生成密钥对

目的：在设备上生成 ECDSA P-256 密钥对，用于后续生成 CSR 和 TLS 握手。

操作步骤：

1. **检查状态**：
   - 查看证书状态卡片，确认当前状态
   - 如果显示「已生成密钥」，跳过此步骤（除非需要重新生成）

2. **点击生成按钮**：点击 **「生成密钥对」** 按钮

3. **阅读说明**：弹出的「生成 HTTPS 密钥对」弹窗显示：

   ```
   为设备生成 ECDSA P-256 密钥对，用于 mTLS 身份验证
   ```

4. **注意覆盖警告**：
   - 如果已存在密钥对，会显示黄色警告：

     ```
     注意：已存在密钥对，继续将覆盖现有密钥！
     ```

   - **慎重考虑**：覆盖密钥后，旧密钥生成的 CSR 和证书将失效
   - 确认要覆盖才继续

5. **开始生成**：点击 **「生成」** 按钮

6. **等待完成**：
   - 生成 ECDSA P-256 密钥约需 3-5 秒
   - 弹窗显示进度或结果信息

7. **确认成功**：
   - 显示「密钥对生成成功」
   - 关闭弹窗

- 状态卡片更新为「已生成密钥，未安装证书」
- 「生成 CSR」「安装证书」「安装 CA」按钮变为可用

技术说明

- **密钥类型**：ECDSA P-256（256 位椭圆曲线密钥）
  - 优势：生成快、加密强度高、密钥尺寸小
  - 标准：NIST P-256 / secp256r1
- **存储位置**：设备 NVS（非易失性存储），重启不丢失
- **用途**：
  - HTTPS 服务器端证书（设备作为服务器）
  - mTLS 客户端证书（设备作为客户端连接其他 mTLS 服务）
- **安全性**：私钥永不导出，仅在设备本地使用

5.3.2 生成 CSR（证书签名请求）

目的：生成 CSR（Certificate Signing Request），提交给 CA（证书颁发机构）签发证书。

前提条件：

- 已完成「5.3.1 生成密钥对」
- 「生成 CSR」按钮已启用（蓝色可点击）

操作步骤：

1. **点击生成 CSR 按钮**：点击 **「生成 CSR」** 按钮

2. **填写 CSR 信息**：在「证书签名请求 (CSR)」弹窗中填写：

字段	必填	说明	填写示例
设备 ID (CN)	否	设备的唯一标识（Common Name），留空使用默认配置	`TIANSHAN-RM01-0001`、`DEVICE-AGX-001`
组织 (O)	否	公司或组织名称（Organization）	`HiddenPeak Labs`、`My Company`
部门 (OU)	否	组织单位或部门（Organizational Unit）	`Device`、`Production`、`IoT`

字段说明：

- **CN（Common Name）**：最重要，建议使用规范命名，如 `前缀-型号-序列号`
- **O 和 OU**：可选，用于证书的组织信息
- **留空处理**：如果所有字段留空，系统使用默认配置（如设备 MAC 地址作为 CN）

3. **生成 CSR**：点击 **「生成 CSR」** 按钮

4. **等待生成**：约 2-3 秒，系统会：
   - 使用私钥和填写的信息生成 CSR
   - CSR 格式符合 X.509 标准（RFC 2986）

5. **查看 CSR**：生成成功后，弹窗会展开显示：

代码块

```
CSR 内容（复制到 CA 服务器签发）
┌──────────────────────────────────────────────────────┐
│ -----BEGIN CERTIFICATE REQUEST-----        │
│ MIIBVTCByQIBADBGMQswCQYDVQQGEwJVUzEWMBQG │
│ A1UECgwNSGlkZGVuUGVhayBMYWJzMR8wHQYDVQ │
│ ...（Base64 编码的 CSR 内容）...          │
│ -----END CERTIFICATE REQUEST-----          │
└──────────────────────────────────────────────────────┘
```

6. **复制 CSR**：点击 **「复制到剪贴板」** 按钮复制 CSR 内容

7. **保存 CSR**（可选）：
   - 将 CSR 粘贴到文本编辑器
   - 保存为 `.csr` 文件（如 `device.csr`）
   - 便于后续提交给 CA

CSR 包含的信息

- 设备的公钥（从密钥对中提取）
- 主体信息（CN、O、OU）
- 设备签名（证明请求者拥有私钥）
- **不包含**私钥（私钥永不离开设备）

下一步

将复制的 CSR 提交给您的 CA 进行签发，详见「5.3.3 提交 CA 签发」。

5.3.3 提交 CA 签发（外部操作）

目的：将 CSR 提交给证书颁发机构（CA），获取签发的设备证书。

注意：此步骤在设备**外部**完成，需要访问 CA 服务器或联系 PKI 管理员。

常见 CA 方式

方式一：企业内部 CA（如 step-ca）

如果您的组织使用 step-ca 或其他内部 CA：

1. **登录 CA 服务器**（或使用 step CLI）
2. **提交 CSR 签发证书**：

代码块

```
# 将 CSR 保存为 device.csr，然后运行
step ca certificate --csr device.csr device.crt

# 或者直接粘贴 CSR 内容
cat > device.csr << 'EOF'
-----BEGIN CERTIFICATE REQUEST-----
（粘贴 CSR 内容）
-----END CERTIFICATE REQUEST-----
EOF

step ca certificate --csr device.csr device.crt
```

3. **获取证书和 CA 链**：
   - `device.crt`：设备证书（下一步需要）
   - `ca-chain.pem`：CA 证书链（从 CA 服务器下载）

方式二：Let's Encrypt（公共 CA）

如果设备有公网域名，可以使用 Let's Encrypt：

1. 使用 ACME 协议自动化签发
2. 需要验证域名所有权（HTTP-01 或 DNS-01 验证）
3. 签发免费 90 天证书

注意：Let's Encrypt 更适合公网服务，内网设备建议使用企业内部 CA。

方式三：手动签发（PKI 管理员）

1. **提交 CSR**：将 CSR 内容发送给 PKI 管理员（邮件、工单系统）
2. **等待签发**：管理员审核后使用 CA 私钥签发证书
3. **获取证书**：接收签发的设备证书（.crt 或 .pem 文件）

等待时间

- **自动化 CA**：秒级到分钟级
- ⏳ **手动 CA**：小时到天级（取决于审批流程）

获取的文件

签发完成后，您应该获得以下文件：

文件	必需	说明	格式
设备证书	是	CA 签发的设备证书	`device.crt`、PEM 格式
CA 证书链	是	根 CA + 中间 CA 证书	`ca-chain.pem`、PEM 格式

准备好这两个文件后，继续「5.3.4 安装设备证书」。

5.3.4 安装设备证书

目的：将 CA 签发的设备证书安装到设备，使 mTLS 功能生效。

前提条件：

- 已生成密钥对
- 已从 CA 获取签发的设备证书（PEM 格式）
- 证书是使用本设备密钥生成的 CSR 签发的

操作步骤：

1. **打开证书文件**：在您的电脑上打开 CA 签发的证书文件（如 `device.crt`）

2. **复制证书内容**：全选并复制证书内容，格式应为：

   代码块

   ```
   -----BEGIN CERTIFICATE-----
   MIICxzCCAa+gAwIBAgIRAMxBxOxxxxxxxxxxxxxxxxxxxxxx
   （多行 Base64 编码内容）
   xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxQ=
   -----END CERTIFICATE-----
   ```

3. **点击安装按钮**：在安全页面点击 **「安装证书」** 按钮

4. **粘贴证书**：在「安装设备证书」弹窗中：
   - 将证书内容粘贴到「证书 PEM」文本框
   - 确保内容完整（包括开头和结尾的 `-----BEGIN/END-----`）

5. **提交安装**：点击 **「安装」** 按钮

6. **等待验证**：系统会自动验证：
   - 证书格式是否正确（PEM 格式）
   - 证书与设备私钥是否匹配（关键）
   - 证书是否在有效期内
   - 证书主体信息是否符合预期

7. **确认成功**：
    - 显示「证书安装成功」
    - 显示证书详情（CN、签发者、有效期等）
    - 状态卡片更新为「已激活」或「已生成密钥，未安装 CA 链」

常见错误处理

错误提示	原因	解决方法
证书与私钥不匹配	证书不是用当前设备的 CSR 签发的	①重新生成 CSR 并提交 CA 签发 ②或删除凭证重新开始
证书格式错误	内容不是有效的 PEM 格式	检查复制内容是否完整，包含 BEGIN/END 标记
证书已过期	CA 签发的证书已过有效期	联系 CA 重新签发有效期内的证书
解析证书失败	证书内容损坏或格式错误	重新从 CA 获取证书文件，确保内容完整

验证证书与私钥匹配的原理

系统会执行以下验证：

1. 从证书中提取公钥
2. 从设备 NVS 中加载私钥
3. 验证公钥和私钥是否为配对的密钥对
4. 只有匹配的证书才能安装成功

为什么重要：不匹配的证书无法用于 TLS 握手，会导致连接失败。

5.3.5 安装 CA 证书链

目的：安装 CA 证书链（根 CA + 中间 CA），使客户端可以验证设备证书的签发者。

前提条件：

- 已安装设备证书（完成 5.3.4）
- 已从 CA 获取证书链文件（PEM 格式）

为什么需要 CA 链：

- 客户端（浏览器、API 客户端）需要验证设备证书是否由可信 CA 签发
- CA 链提供完整的信任路径：`设备证书 ← 中间 CA ← 根 CA`
- 如果缺少 CA 链，客户端会提示「无法验证证书」

操作步骤：

1. **准备 CA 链文件**：
   - 文件名如 `ca-chain.pem`、`root-ca.pem` 等
   - 内容包含根 CA 和中间 CA 证书（可以拼接在一个文件中）

2. **打开并复制 CA 链**：全选并复制证书链内容，格式为：

   代码块

   ```
   -----BEGIN CERTIFICATE-----
   （根 CA 证书 - Base64 内容）
   -----END CERTIFICATE-----
   -----BEGIN CERTIFICATE-----
   （中间 CA 证书 - Base64 内容）
   -----END CERTIFICATE-----
   ```

说明：

- 可以包含多个证书块（根 CA、一个或多个中间 CA）

- 顺序不重要（系统会自动识别根 CA 和中间 CA）
- 如果只有一个 CA（简化环境），只需一个证书块

3. **点击安装 CA 按钮**：点击 **「安装 CA」** 按钮

4. **粘贴证书链**：在「安装 CA 证书链」弹窗中：
   - 将 CA 链内容粘贴到文本框
   - 检查内容完整（包括所有 BEGIN/END 标记）

5. **提交安装**：点击 **「安装」** 按钮

6. **等待完成**：系统验证证书链格式并存储到 NVS

7. **确认成功**：
   - 显示「CA 链安装成功」
   - 状态卡片更新为「已激活」（如果证书有效）
   - mTLS 配置完成

CA 链示例

单级 CA（简化环境）：

代码块

```
-----BEGIN CERTIFICATE-----
（自签名根 CA）
-----END CERTIFICATE-----
```

两级 CA（标准企业环境）：

代码块

```
-----BEGIN CERTIFICATE-----
（根 CA 证书）
```

```
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
（中间 CA 证书）
-----END CERTIFICATE-----
```

安装后验证

1. **检查状态卡片**：
   - 状态应为「已激活」
   - 有效状态显示「有效」
   - 剩余天数应显示正常值

2. **测试 mTLS 连接**（需要客户端配置）：
   - 使用支持 mTLS 的客户端连接设备
   - 验证双向认证是否成功

5.4 查看已安装的证书

用途：

- 查看设备证书的完整 PEM 内容
- 确认证书信息（CN、有效期等）
- 复制证书分享给其他系统（如配置客户端信任）

操作步骤：

1. **前提条件**：已安装设备证书（「查看证书」按钮为可用状态）

2. **点击查看按钮**：点击 **「查看证书」** 按钮

3. **等待加载**：弹窗显示「加载中...」，约 1-2 秒

4. **查看证书内容**：
   - 显示完整的设备证书 PEM 内容
   - 可滚动查看所有内容

5. **复制证书**（可选）：
   - 点击 **「复制到剪贴板」** 复制证书
   - 用途：配置客户端、存档备份

6. **关闭弹窗**：点击「关闭」按钮

证书内容解读

证书 PEM 内容为 Base64 编码的 X.509 证书，可以使用工具解析查看详细信息：

在 Linux/macOS 上解析证书：

代码块

```
# 将证书保存为 device.crt，然后运行
openssl x509 -in device.crt -text -noout
```

输出示例：

代码块

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 01:23:45:67:89:ab
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: CN=TianShanOS Intermediate CA
        Validity
```

```
            Not Before: Jan 27 00:00:00 2026 GMT
            Not After : Jan 27 00:00:00 2027 GMT
        Subject: CN=TIANSHAN-RM01-0001, O=HiddenPeak Labs, OU=Device
        Subject Public Key Info:
            Public Key Algorithm: id-ecPublicKey
                Public-Key: (256 bit)
```

5.5 删除所有 PKI 凭证（工厂重置）

警告：此操作会删除所有 HTTPS 证书相关数据，执行前请确认。

删除内容

点击「删除凭证」会删除以下所有内容：

- 设备私钥（ECDSA P-256）
- 设备证书
- CA 证书链

何时使用

场景	说明
设备重新配置	需要更换设备 ID 或证书体系
证书体系变更	公司更换 CA 或 PKI 架构调整
测试环境重置	清空测试数据，重新测试证书流程
安全事件处理	怀疑私钥泄露，需要重新生成

操作步骤

1. **点击删除按钮**：点击 **「删除凭证」** 按钮（红色）

2. **阅读警告**：系统弹出确认对话框：

   代码块

   ```
   警告：删除所有 PKI 凭证

   此操作将删除：
   • 设备私钥
   • 设备证书
   • CA 证书链

   删除后：
   • mTLS 功能将不可用
   • 需要重新生成密钥和申请证书
   • 此操作不可恢复

   确定要继续吗？
   ```

3. **确认删除**：
   - 确认理解后果后，点击「确定」
   - 如果不确定，点击「取消」

4. **验证结果**：
   - 状态卡片更新为「未生成密钥」
   - 所有按钮重置为初始状态
   - 「密钥管理」表格中 HTTPS 密钥行消失

删除后的影响

- **HTTPS 服务**：
  - 可能降级为 HTTP（如果配置允许）

    - 或使用临时自签名证书
    - mTLS 双向认证失败

- **客户端连接**：
    - 使用 mTLS 的客户端无法验证设备身份
    - 浏览器可能显示证书错误

- **恢复方法**：
    - 重新执行「5.3 完整证书申请流程」
    - 从步骤 5.3.1 重新开始

5.6 证书续期

何时需要续期

- 证书即将过期（建议提前 **30 天**开始续期）
- 状态卡片显示「即将过期」或剩余天数 < 30
- 根据公司 PKI 策略定期续期（如每年）

续期流程

方式一：保留密钥，仅续期证书（推荐）

如果私钥未泄露，可以保留密钥对，仅更新证书：

1. **生成新 CSR**：
    - 点击「生成 CSR」（可以使用相同的 CN）
    - 复制 CSR 内容

2. **提交 CA 签发**：
    - 将新 CSR 提交给 CA
    - 获取新签发的证书

3. **安装新证书**：
    - 点击「安装证书」
    - 粘贴新证书内容
    - 系统会自动覆盖旧证书

4. **验证状态**：
    - 检查状态卡片，「过期时间」应更新为新日期
    - 「剩余天数」应重置为新证书的有效期

方式二：完全重新生成（密钥泄露时）

如果怀疑私钥泄露，需要重新生成所有凭证：

1. **删除旧凭证**：点击「删除凭证」
2. **重新生成密钥对**：从「5.3.1」开始
3. **重新申请证书**：执行完整的 5.3 流程

续期注意事项

- 续期前可以继续使用旧证书（直到过期）
- 新证书安装后立即生效，旧证书自动失效
- CA 链通常不需要更新（除非 CA 体系变更）
- 如果 CA 链也过期，需要同时更新 CA 链

自动化续期（未来功能）

说明：当前版本需要手动续期。未来版本可能支持：

- 自动检测证书即将过期
- 自动生成 CSR 并提交 ACME CA
- 自动安装续期后的证书

任务五：配置包 (Config Pack)

「配置包」区块用于在设备间安全地分发配置：将配置文件加密、签名后打包为 `.tscfg` 文件，只有持有对应设备证书的目标设备才能解密使用。适用于批量部署、远程下发配置等场景。

5.5.1 配置包功能概览

功能	说明	权限
导出设备证书	导出本设备的 HTTPS 证书（PEM）及指纹，供他人向本设备发送加密配置包	admin
导入配置包	上传或粘贴 .tscfg 文件，验证签名后解密并保存到设备（加密存储）	admin
导出配置包	选择本地文件、填写目标设备证书，生成加密的 .tscfg 包发给目标设备	仅 Developer 设备
查看配置包列表	按目录查看已存在的配置包文件及签名者、官方标记、状态	admin

状态卡片：区块顶部显示当前设备类型、证书 CN、证书指纹、格式版本等；若尚未配置 mTLS 证书，部分功能可能不可用。

5.5.2 导出设备证书

用途：将本设备的证书发给需要向您发送加密配置的开发者或系统，以便对方在「导出配置包」时粘贴您的证书，生成只有本设备能解密的配置包。

步骤：

1. 点击 **「导出设备证书」**
2. 弹窗中查看证书指纹 (SHA256)、证书 CN、证书 PEM
3. 点击 **「复制到剪贴板」** 或自行复制 PEM 内容，通过安全渠道发送给对方

5.5.3 导入配置包

用途：接收他人或系统下发的 .tscfg 配置包，验证签名并解密后保存到设备。

步骤：

1. 点击 **「导入配置包」**
2. 选择 .tscfg 文件，或粘贴 JSON 内容
3. 可先点击 **「仅验证」** 检查签名与内容，再点击 **「导入」** 写入设备

说明：「仅验证」只校验配置包的签名和格式，**不会**将配置写入设备，适合在正式导入前确认来源与内容是否可信。

4. 导入成功后，配置包会加密存储在设备上，可通过 `config.pack.content` 等 API 按需解密使用

5.5.4 导出配置包（仅 Developer 设备）

用途：选择本设备上的配置文件，用目标设备的证书加密并签名，生成 .tscfg 文件，供目标设备导入。

步骤：

1. 若设备非 Developer 类型，该按钮会提示「仅 Developer 设备可导出配置包」
2. 点击 **「导出配置包」**，在弹窗中浏览并选择要打包的配置文件（可多选）
3. 填写配置名称、描述，粘贴**目标设备**导出的证书 PEM
4. 点击 **「生成配置包」**，将生成的 .tscfg 内容复制或下载后发送给目标设备

5.5.5 查看配置包列表

用途：按指定目录（如 `/sdcard/config`）列出已存在的配置包文件，查看文件名、大小、签名者、是否官方、状态等。

步骤：点击 **「查看配置包列表」**，输入或使用默认目录路径，点击 **「刷新」** 即可查看列表。

列表字段说明：

- **签名者**：对该配置包进行签名的证书 CN，用于验证来源
- **官方**：若签名来自系统信任的官方证书，会标记为「官方」，便于区分厂商下发的配置与自签名配置
- **状态**：该配置包是否已应用、是否有效等

安全存储 vs 文件密钥对比

帮助您理解为什么 TianshanOS 默认使用安全存储（NVS）而非文件。

对比表格

特性	安全存储（NVS）	文件密钥（SD 卡）
私钥保护	硬件 NVS 加密分区，难以提取	明文 PEM 文件，易被复制、泄露

使用便捷性	只需记住密钥 ID，无需管理文件路径	需要记住文件路径，容易混淆
密钥备份	需勾选「可导出」才能备份	可直接复制文件备份
跨设备使用	密钥绑定单个设备	可复制到多个设备使用
防止意外删除	需显式删除操作	文件系统错误可能导致丢失
存储容量	NVS 有限（约 8-32 个密钥）	SD 卡容量大
访问速度	快速（NVS 读取）	较慢（SD 卡 I/O）
安全性评分	高（5 星）	较低（2 星）
推荐场景	**生产环境、长期使用、安全敏感**	临时使用、密钥迁移、多设备共享

安全存储的优势

1. **硬件级保护：**
   - NVS 分区支持 Flash 加密（生产环境可启用）
   - 私钥难以通过物理方式提取

2. **默认安全：**
   - 私钥默认不可导出（除非显式勾选）
   - 防止误操作导致泄露

3. **集中管理：**
   - 所有密钥在 WebUI 统一查看和管理
   - 不会散落在文件系统各处

4. **权限控制：**
   - 可设置「隐藏密钥」，防止低权限用户访问
   - 配合「可导出」标记，实现细粒度控制

何时使用文件密钥

尽管安全存储更安全，以下场景可以使用文件密钥：

- **密钥迁移**：从其他设备导入已有密钥
- **临时测试**：临时使用，用完即删
- **多设备共享**：同一个密钥需要在多个设备使用
- **外部工具**：需要在桌面 SSH 客户端（如 PuTTY、Terminal）使用

使用文件密钥的 CLI 命令：

代码块

```
# 使用 SD 卡上的密钥文件连接
ssh --host 192.168.1.100 --user nvidia --key /sdcard/id_rsa --shell
```

常见问题与故障排查

7.1 密钥管理问题

Q1：生成密钥时提示"存储空间已满"？

现象：点击「生成」后显示错误「NVS storage full」或「密钥数量已达上限」。

原因：NVS 分区容量有限（48KB），最多存储：

- 约 8 个 RSA-4096 密钥
- 约 12 个 RSA-2048 密钥
- 约 32 个 ECDSA P-256 密钥

解决方法：

1. 进入「密钥管理」列表，查看已有密钥
2. 删除不再需要的密钥（先撤销已部署的主机）
3. 删除后重新生成新密钥

建议：

- 定期清理过期密钥
- 优先使用 ECDSA P-256（占用空间小）
- 避免生成过多 RSA-4096 密钥

Q2：ECDSA 密钥无法连接 SSH 服务器？

现象：使用 ECDSA 类型密钥部署或连接时失败，错误提示「认证失败」或「不支持的密钥类型」。

原因：当前版本 TianshanOS 的 SSH 客户端暂不支持 ECDSA 密钥的公钥认证（已知限制）。

解决方法：

1. 删除该 ECDSA 密钥
2. 重新生成密钥时选择 **RSA 2048-bit** 或 **RSA 4096-bit**
3. 重新部署公钥

说明：

- ECDSA 密钥本身是安全的（用于 HTTPS 证书没有问题）
- SSH 公钥认证的 ECDSA 支持将在未来版本中添加

Q3：如何备份密钥？

场景：担心设备损坏或固件升级丢失密钥，需要备份。

方法一：导出私钥（需提前设置）

1. 生成密钥时勾选 **「允许导出私钥」**
2. 生成后，点击「私钥」按钮
3. 下载为 `.pem` 文件
4. 存储在安全的离线介质（加密 U 盘、密码管理器）

方法二：记录密钥 ID 和配置

1. 记录密钥 ID、类型、备注
2. 记录部署到哪些服务器
3. 如果密钥丢失，重新生成并重新部署

推荐：

- 重要生产环境密钥：勾选「可导出」并备份
- 测试环境密钥：不备份，丢失后重新生成

Q4：删除密钥后还能恢复吗？

答案：无法恢复。

删除操作是永久性的：

- 密钥从 NVS 完全清除
- 没有回收站或撤销功能
- 唯一恢复方式：之前导出的私钥备份

删除前的检查清单：

1. 确认该密钥没有部署到重要服务器
2. 如果有部署，已完成撤销或移除
3. 如果需要备份，已导出私钥
4. 已通知其他可能使用该密钥的人员

误删后的补救：

- 如果有备份：通过 CLI 的 `key --import` 命令重新导入
- 如果无备份：重新生成新密钥，重新部署到所有服务器

7.2 部署与连接问题

Q5：部署公钥时提示"连接超时"？

现象：点击「开始部署」后，长时间等待，最终显示「Connection timeout」。

可能原因：

1. 网络不通（设备无法访问目标主机）
2. 目标主机未开启 SSH 服务
3. 防火墙阻止 22 端口
4. IP 地址或端口填写错误

排查步骤：

步骤 1：检查网络连通性

- 在终端中 ping 目标主机（如果有 root 权限）
- 或在其他设备上 ping 该主机，确认主机在线

步骤 2：检查 SSH 服务

- 在目标服务器上运行：`systemctl status sshd`（或 `service ssh status`）
- 确认 SSH 服务正在运行

步骤 3：检查防火墙

- 设备防火墙：确认允许出站 SSH 连接
- 服务器防火墙：确认允许入站 22 端口（或自定义端口）

代码块

```
# 检查防火墙状态（服务器端）
sudo ufw status
sudo firewall-cmd --list-all
```

步骤 4：验证参数

- 确认 IP 地址、端口、用户名填写正确
- 尝试从其他设备 SSH 连接，验证服务器可访问性

Q6：部署时提示"密码错误"？

现象：输入密码后提示「Authentication failed」或「密码错误」。

可能原因：

1. 密码输入错误（大小写、特殊字符）
2. 用户名错误
3. SSH 配置禁用密码认证
4. 用户账户被锁定

解决方法：

1. **重新确认密码**：
   - 注意大小写（Linux 密码区分大小写）
   - 注意特殊字符（空格、引号等）
   - 在其他地方测试密码是否正确

2. **检查用户名**：
   - 确认用户名拼写正确
   - 某些系统 root 用户可能禁用密码登录

3. **检查 SSH 配置**（服务器端）：

代码块

```
# 查看 SSH 配置
sudo cat /etc/ssh/sshd_config | grep PasswordAuthentication

# 确保以下配置为 yes
PasswordAuthentication yes

# 修改后重启 SSH 服务
sudo systemctl restart sshd
```

4. **检查用户状态**：

代码块

```
# 查看用户是否被锁定
sudo passwd -S <username>
```

Q7：部署成功但仍需要密码？

现象：部署显示成功，但使用密钥连接时仍然提示输入密码。

可能原因：

1. `authorized_keys` 文件权限不正确
2. SSH 配置未启用公钥认证
3. 主目录权限过于宽松
4. SELinux 上下文错误（RHEL/CentOS）

解决方法：

1. **检查文件权限**（在服务器端执行）：

   代码块

   ```
   # 检查权限
   ls -ld ~/.ssh
   ls -l ~/.ssh/authorized_keys

   # 正确的权限应该是:
   # drwx------ (700) ~/.ssh
   # -rw------- (600) ~/.ssh/authorized_keys

   # 如果权限不对，修正:
   chmod 700 ~/.ssh
   chmod 600 ~/.ssh/authorized_keys
   ```

2. **检查文件内容：**

   代码块

   ```
   # 查看 authorized_keys 内容
   cat ~/.ssh/authorized_keys

   # 应该包含以 ssh-rsa 或 ecdsa-sha2- 开头的公钥行
   ```

3. **检查 SSH 配置**（服务器端）：

   代码块

```
sudo cat /etc/ssh/sshd_config | grep -E "PubkeyAuthentication|AuthorizedKeysFile"

# 确保以下配置
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
```

4. **检查 SELinux**（RHEL/CentOS）：

代码块

```
# 恢复 SELinux 上下文
restorecon -R ~/.ssh
```

5. **重启 SSH 服务**（如修改了配置）：

代码块

```
sudo systemctl restart sshd
```

Q8：「已部署主机」列表为空？

现象：已经部署过公钥，但列表显示「暂无已部署主机」。

原因：只有通过 WebUI「部署」功能部署的主机会自动添加到此列表。

不会出现在列表中的情况：

- 通过终端 CLI 的 `ssh --copyid` 部署
- 手动复制公钥到服务器的 `authorized_keys`
- 其他设备或工具部署的公钥

解决方法：

- 如果确实需要在 WebUI 中管理，可重新通过 WebUI 的「部署」功能部署一次；或使用「导入主机」导入包含该主机信息的 .tscfg 配置包，使列表与实际情况一致
- 或者仅通过 CLI 的 `hosts --deployed` 命令查看和管理

设计说明：

- 列表记录存储在设备本地（`ts_ssh_hosts_config`）
- 仅记录通过 WebUI 或特定 API 部署的主机
- 目的是提供统一的部署管理界面

Q8b：导入主机或导入配置包时提示「签名验证失败」？

现象：点击「导入主机」或「导入配置包」并选择/粘贴 .tscfg 后，提示签名验证失败或无法解密。

可能原因：

- .tscfg 文件损坏或复制不完整（缺少头尾、被截断）
- 配置包不是用当前设备（或目标设备）所信任的证书签发的
- 设备系统时间偏差过大，导致证书有效期校验失败
- 粘贴的 JSON 格式错误（多余空格、缺少引号等）

解决方法：

1. **检查文件完整性**：重新从来源获取 .tscfg，确保包含完整的 `-----BEGIN ...-----` / `-----END ...-----` 等块，或重新导出
2. **确认签发来源**：确认该配置包是由本机证书或您信任的官方/目标设备证书签发的；导入主机时若为「自加密」导出，需在同一台或拥有同一证书的设备上导入
3. **校对设备时间**：在系统/网络页检查设备日期时间是否正确，必要时同步 NTP
4. **仅验证**：在配置包导入弹窗中先点击「仅验证」，查看具体报错信息（如签名者、证书 CN），再根据提示排查

7.3 已知主机指纹问题

Q9：「已知主机指纹」列表为空？

现象：打开安全页面，列表显示「暂无已知主机」，但已经连接过 SSH 服务器。

可能原因：

原因 1：确实没有连接过

- 解决方法：执行一次 SSH 连接（部署公钥或使用密钥连接），信任主机后列表会有记录

原因 2：模块未初始化（已知 Bug）

- 现象：已连接并信任主机，但列表仍为空；刷新页面后突然显示
- 原因：`ts_known_hosts` 模块采用延迟初始化，安全服务启动时未主动初始化
- 解决方法：
  a. 执行一次 SSH 连接（触发初始化）
  b. 刷新安全页面
  c. 列表应该会显示记录

原因 3：NVS 数据损坏

- 现象：终端 CLI `hosts --list` 也为空
- 解决方法：重新连接并信任主机，重建记录

如何确认是否有指纹记录（需要 root 权限访问终端）：

代码块

```
# 查看已知主机列表
hosts --list

# 查看详细信息
```

```
hosts --list --json
```

Q10：主机指纹不匹配警告如何处理？

现象：连接时弹出红色警告「安全警告：主机指纹不匹配」。

正确处理流程（详见 4.4 章节）：

1. **立即停止**：不要盲目点击「更新主机密钥」
2. **联系管理员**：确认服务器是否重装或更换密钥
3. **确认合法后**：点击「更新主机密钥」
4. **无法确认时**：点击「取消」，排查网络安全

错误示范：

- 看到警告就点「更新主机密钥」（可能中招中间人攻击）
- 关闭警告继续连接（系统会阻止）
- 忽视警告，认为是系统 Bug（这是重要的安全特性）

安全提示：

- 主机指纹不匹配是**严重的安全事件**
- 必须核实后才能继续
- 如果频繁出现，可能网络环境不安全

Q11：如何验证主机指纹是否正确？

场景：首次连接或指纹不匹配时，需要验证服务器指纹的真实性。

方法一：在服务器端获取官方指纹

1. **登录服务器**（通过控制台或其他可信方式）

2. **查看主机密钥指纹**：

代码块

```
# ECDSA 密钥（推荐）
ssh-keygen -lf /etc/ssh/ssh_host_ecdsa_key.pub

# ED25519 密钥（更安全，较新）
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

# RSA 密钥（兼容性好）
ssh-keygen -lf /etc/ssh/ssh_host_rsa_key.pub
```

3. **输出示例**：

代码块

```
256 SHA256:xYzAbC123456789aBcDeF/gHiJkLmNoPqRsTuVwXyZ01= root@server (ECDSA)
```

4. **对比指纹**：
   - 将 `SHA256:xYz...` 部分与设备显示的指纹对比
   - 必须**完全一致**（逐字符对比）

方法二：联系服务器管理员

1. 联系负责该服务器的管理员
2. 要求提供官方主机密钥指纹（SHA256 格式）
3. 对比确认

方法三：使用其他可信设备验证

1. 使用已确认安全的其他电脑或设备
2. SSH 连接该服务器，记录指纹
3. 对比与 TianshanOS 显示的指纹是否一致

安全提示：

- 高安全场景（生产环境、金融、医疗）建议首次连接前验证指纹
- 内部测试环境可以适当简化流程

7.4 HTTPS 证书问题

Q12：生成 CSR 按钮灰色不可用？

现象：「生成 CSR」按钮显示为灰色，无法点击。

原因：尚未生成密钥对。CSR 需要使用私钥签名，因此必须先有密钥对。

解决方法：

1. 点击 **「生成密钥对」** 按钮
2. 等待密钥生成成功
3. 「生成 CSR」按钮会自动变为可用（蓝色）

相关按钮：

- 「安装证书」、「安装 CA」也需要先生成密钥对
- 只有「生成密钥对」和「删除凭证」在任何状态下都可用

Q13：安装证书时提示"证书与私钥不匹配"？

现象：粘贴证书内容后点击「安装」，显示错误「Certificate and private key do not match」。

原因：

- 该证书不是使用**当前设备密钥**生成的 CSR 签发的
- 可能是：
  - 使用了其他设备的 CSR
  - 在生成 CSR 后又重新生成了密钥对（导致私钥变化）
  - CA 签发时使用了错误的 CSR

解决方法：

方法一：重新生成 CSR 并签发

1. 在设备上重新点击「生成 CSR」
2. 复制新的 CSR 内容
3. 提交给 CA 重新签发
4. 安装新签发的证书

方法二：删除凭证重新开始

1. 点击「删除凭证」
2. 从「5.3.1 生成密钥对」开始重新执行完整流程
3. 确保 CSR、签发、安装使用的是同一个密钥对

预防措施：

- 生成 CSR 后不要重新生成密钥对
- 签发前确认 CSR 来源正确
- 标记好文件名（如 `device-001.csr`、`device-001.crt`）

Q14：证书状态显示"即将过期"？

现象：状态卡片显示「即将过期」，剩余天数 < 30。

处理建议：

- **30-15 天**：开始准备续期，不紧急
- **15-7 天**：尽快续期，避免影响服务
- **< 7 天**：立即续期，高优先级

续期流程（保留密钥）：

1. **生成新 CSR**：
   - 点击「生成 CSR」
   - 可以使用相同的 CN（设备 ID）
   - 复制 CSR 内容
2. **提交 CA 签发**：
   - 将 CSR 提交给您的 CA
   - 说明是续期操作（可能加快审批）
3. **安装新证书**：
   - 获取新签发的证书后
   - 点击「安装证书」
   - 粘贴新证书内容
   - 旧证书自动被覆盖
4. **验证续期**：

    - 检查「过期时间」已更新为新日期
    - 检查「剩余天数」已重置

注意事项：

- 续期期间旧证书仍可用（直到过期）
- 安装新证书后立即生效
- CA 链通常不需要更新（除非 CA 本身过期）
- 如果 CA 链也过期，需要同时更新 CA 链

Q15：删除凭证后 HTTPS 服务是否还能用？

答案：取决于系统配置，可能出现以下情况。

情况 1：降级为 HTTP

- HTTPS 服务关闭，WebUI 通过 HTTP 访问（端口 80）
- 安全性降低，数据明文传输

情况 2：使用自签名证书

- 系统自动生成临时自签名证书
- 浏览器会显示「证书不可信」警告
- 需要手动信任才能访问

情况 3：服务不可用

- 如果配置要求必须使用 mTLS，HTTPS 服务可能拒绝连接
- 需要重新安装证书才能恢复

建议：

- 删除凭证前先确认系统配置和备用方案

- 测试环境可以放心删除
- 生产环境删除前建议先安装新证书，避免服务中断

7.5 权限与认证问题

Q16：admin 用户可以使用安全页面的所有功能吗？

答案：是的。

admin 权限范围：

- 安全页面的所有功能
- 系统、网络、文件页面
- 终端、自动化、指令页面需要 root 权限，admin 不可访问

安全页面功能清单（admin 可用）：

- 生成、删除、导出 SSH 密钥
- 部署公钥到远程服务器
- 撤销已部署的公钥
- 管理已部署主机记录（含导入主机配置）
- 查看和删除已知主机指纹
- 生成、安装、删除 HTTPS 证书
- 生成 CSR、安装 CA 链
- 配置包：导出设备证书、导入配置包、查看配置包列表

如果需要 root 权限功能：

- 请联系系统管理员，使用 root 账户登录
- 或申请升级到 root 权限

Q17：导出私钥时提示"权限不足"或按钮灰色？

现象：点击「私钥」按钮时：

- 按钮显示为灰色（disabled），无法点击
- 或点击后提示「此密钥不可导出私钥」

原因：该密钥生成时**未勾选**「允许导出私钥」。

TianshanOS 安全设计：

- 默认情况下，私钥**不可导出**（安全最佳实践）
- 只有显式勾选「允许导出私钥」的密钥才能导出私钥
- 此限制与用户权限（admin/root）无关，是密钥自身的属性

解决方法：

方法一：生成新的可导出密钥

1. 生成新密钥时勾选 **「允许导出私钥」**
2. 生成后可以导出私钥备份
3. 部署到需要的服务器

方法二：仅导出公钥

- 如果不需要私钥，可以导出公钥（所有密钥都可导出公钥）
- 公钥可安全分享，没有泄露风险

方法三：接受限制

- 如果该密钥已部署到生产服务器，保持不可导出状态反而更安全
- 通过 WebUI「部署」功能管理，无需导出私钥

设计理念：

- 私钥是最敏感的安全凭证，应尽量避免导出
- 仅在确实需要备份或迁移时才导出
- 默认不可导出，减少误操作泄露风险

最佳实践建议

8.1 密钥管理最佳实践

1. 密钥命名规范

推荐命名方式：

- `<用途>_<环境>`：如 `agx_prod`、`lpmu_test`
- `<服务器>_<用户>`：如 `server01_root`、`gateway_admin`
- `<项目>_<序号>`：如 `project_a_01`、`iot_device_001`

避免的命名：

- `key1`、`test`、`temp`（无意义）
- 中文或特殊字符（可能导致兼容性问题）
- 过长的名称（不便于终端输入）

2. 密钥类型选择

推荐策略：

- **日常使用**：RSA 2048-bit（兼容性好、生成快）
- **高安全场景**：RSA 4096-bit（军事级加密，生成慢）
- **ECDSA**：当前仅用于 HTTPS 证书，不用于 SSH

容量规划：

- RSA 2048：可存储约 12 个
- RSA 4096：可存储约 8 个
- 混合使用：根据服务器重要性选择

3. 私钥导出控制

推荐策略：

- **生产环境密钥**：不勾选「允许导出」，永久保存在设备
- **备份密钥**：勾选「允许导出」，定期备份到离线介质
- **测试密钥**：不勾选，丢失后重新生成

导出私钥的安全清单：

- [ ] 确认导出是必要的（备份、迁移）
- [ ] 使用加密传输方式（加密 U 盘、VPN）
- [ ] 存储在安全位置（密码管理器、加密磁盘）
- [ ] 使用后立即删除临时文件
- [ ] 定期审查谁拥有私钥副本

4. 定期审查密钥

建议频率：

- **每季度**：检查密钥列表，删除不再使用的密钥
- **每半年**：审查已部署主机，撤销不需要的访问
- **每年**：考虑轮换重要服务器的密钥

审查检查项：

- [ ] 哪些密钥长期未使用？
- [ ] 哪些密钥部署到已停用的服务器？
- [ ] 是否有测试密钥忘记删除？

- [ ] 存储空间是否接近上限？

8.2 部署管理最佳实践

1. 部署前记录

建议使用电子表格或文档记录：

密钥 ID	部署到服务器	用户名	用途	部署日期	负责人
agx_prod	192.168.1.100	nvidia	AGX 监控	2026-01-30	张三
lpmu_test	10.10.99.50	root	LPMU 测试	2026-01-28	李四

好处：

- 便于后续维护和权限审计
- 人员变动时快速交接
- 故障排查时快速定位

2. 部署后立即测试

流程：

1. 部署成功后，点击「测试」按钮
2. 确认连接成功
3. 如果失败，立即排查，不要留到以后

测试内容：

- 连接是否成功
- 用户名是否正确
- 是否可以执行命令

3. 及时撤销不需要的访问

场景：

- 项目结束，不再需要访问
- 人员离职，撤销个人密钥
- 服务器下线或迁移

操作：

1. 在「已部署主机」列表点击「撤销」
2. 或者在服务器端手动删除 `authorized_keys` 中的对应行

4. 密钥轮换计划

推荐策略：

- **关键服务器**：每年轮换一次密钥
- **一般服务器**：每 2-3 年或按需轮换
- **测试环境**：项目结束后轮换

轮换流程：

1. 生成新密钥（新的 ID，如 `agx_2027`）
2. 部署新密钥到所有服务器
3. 测试新密钥连接
4. 撤销旧密钥
5. 删除旧密钥

8.3 主机指纹管理最佳实践

1. 首次连接验证

标准流程（高安全环境）：

1. 在连接前从管理员获取官方指纹
2. 连接时对比系统显示的指纹
3. 确认一致后输入 `yes` 信任
4. 记录信任时间和验证人员

简化流程（内部测试环境）：

1. 确认 IP 地址正确
2. 直接输入 `yes` 信任
3. 后续依赖自动验证机制

2. 指纹不匹配处理原则

铁律：永远不要忽视指纹不匹配警告

处理优先级：

1. **第一优先**：核实服务器是否重装/更换密钥
2. **第二优先**：排查网络环境（是否在不安全网络）
3. **确认后**：才更新主机密钥

记录变更：

- 记录谁更新了指纹、原因、时间
- 留存服务器重装或密钥更换的证据

3. 定期清理过期记录

清理对象：

- 已停用的服务器
- 已迁移的 IP 地址
- 测试环境的临时服务器

操作：

- 每季度审查「已知主机指纹」列表
- 删除不再使用的主机记录
- 保持列表整洁，便于管理

8.4 证书管理最佳实践

1. 提前续期

续期时间表：

- **证书有效期 > 30 天**：无需操作，定期检查
- **30-15 天**：准备续期流程，联系 CA 或准备文档
- **15-7 天**：执行续期，获取新证书并安装
- **< 7 天**：紧急续期，高优先级

自动提醒（建议）：

- 设置日历提醒（证书到期前 30 天）
- 或使用监控系统检查证书状态

2. 备份 CA 证书链

为什么备份：

- CA 链在所有设备上相同（同一 PKI 体系）
- 备份后配置其他设备时无需重新获取
- CA 服务器故障时仍可使用

备份方式：

1. 点击「查看证书」获取设备证书
2. 将 CA 链单独保存为 `ca-chain.pem`
3. 存储在安全的文档库或配置管理系统

使用备份：

- 配置新设备时直接使用
- CA 服务器不可用时应急使用

3. 测试环境先行

证书更新流程（多设备环境）：

1. 在测试设备上先执行完整流程
2. 验证新证书工作正常
3. 验证 mTLS 连接成功
4. 再推广到生产设备

测试项：

- [ ] 证书安装成功
- [ ] HTTPS 服务正常启动
- [ ] mTLS 双向认证成功
- [ ] 客户端可以访问

4. 定期检查证书状态

检查频率：

- **每月**：打开安全页面，查看状态卡片
- **每季度**：详细检查证书信息，确认无异常
- **重大变更后**：验证证书仍然有效

检查项：

- [ ] 状态是否为「已激活」
- [ ] 有效期是否充足（> 30 天）
- [ ] CN 是否正确
- [ ] 签发者是否为预期 CA

参考文档链接

以下文档提供更深入的技术细节和故障排查，部署到 Mintlify 后可通过站内链接访问。

深入技术细节

- **《安全实现》**：
  - 安全等级定义（L1-L4）
  - 密钥存储原理（NVS 加密、容量规划）
  - SSH 认证机制（密码、公钥、安全存储）
  - WebUI 认证系统（用户体系、权限级别、会话管理）

- **《PKI 设计》**：
  - mTLS 架构设计
  - CA 层级结构（Root CA、Intermediate CA）
  - 证书类型与策略

- 设备激活流程
- 证书更新机制

命令行操作

- **《命令参考》**：
  - `key` 命令：密钥管理的所有 CLI 操作
  - `ssh` 命令：SSH 连接、部署、撤销的完整参数
  - `hosts` 命令：已知主机和已部署主机的 CLI 管理
  - `cert` 命令（如有）：证书管理的 CLI 操作

故障排查

- **《故障排除》**：
  - SSH 连接问题调试
  - 已知主机列表初始为空的解决方法
  - 证书验证失败的排查步骤
  - 网络连接问题诊断

快速入门

- **《快速入门》**：
  - TianshanOS 整体介绍
  - WebUI 页面导航
  - 安全页面简要说明
  - 初次配置向导

结语

本文档涵盖了 TianshanOS 安全页面的所有功能和操作方法。作为 **admin 用户**，您可以使用这些功能管理 SSH 密钥、远程主机连接和 HTTPS 证书。

建议您：

1. 首次使用前完整阅读本文档
2. 在测试环境练习各项操作
3. 将本文档加入书签，便于随时查阅
4. 根据需要查阅相关技术文档深入学习

安全提醒：

- 妥善保管密钥和证书
- 重视所有安全警告（主机指纹不匹配等）
- 定期审查和更新密钥、证书
- 记录重要操作和变更

如有问题或建议，请联系系统管理员或查阅项目文档库。

Comprehensive Guide to Security Page (admin user)

This document provides **users with admin privileges** a complete operating guide for the TianshanOS security page, covering detailed usage methods for all functions such as SSH key management, remote host connection, and HTTPS certificate configuration.

Overview and Preparation

Function of the Security Page

Security Page is the security management center of TianshanOS, centrally managing the following functions:

1. **SSH Key Management**: Generate or import SSH keys on the device, securely stored in the NVS encrypted partition
2. **Remote Host Deployment**: Deploy the public key to a remote Linux server to achieve passwordless login
3. **Known Host Fingerprint** : Records the SSH server fingerprint to prevent man-in-the-middle attacks
4. **HTTPS Certificate Management**: Configure the device's mTLS certificate for mutual authentication

Permissions Scope of Admin User

As an **admin user** , you can use **all features** of the security page, including:

- Generate, delete, and export SSH keys
- Deploy the public key to the remote server
- Revoke the deployed public key
- Manage known host fingerprints
- Generate and manage HTTPS certificates
- Manage configuration packages (import, export, view)

Note: The following pages require **root privileges**, and cannot be accessed by admin users:

- Terminal Page
- Automation Page
- Instruction Page

How to Access the Security Page

1. Access the device's WebUI in the browser (e.g., `http://192.168.4.1`)
2. Log in using the admin account (default password: `rm01`, **it is strongly recommended to change it immediately after the first login**)
3. Click on the **"Security"** link in the top navigation bar
4. Enter the security page (URL route is `#/security`)

Overview of Page Structure

The security page is divided into five sections from top to bottom:

Block	Function Overview
Key Management	Generate, view, export, deploy, revoke, and delete SSH keys
Deployed Hosts	Manage the list of remote servers with deployed public keys; support importing host configurations; test connections, revoke public keys, and remove records
Known host fingerprint	View and delete SSH server fingerprints to prevent man-in-the-middle attacks
HTTPS Certificate	Generate device certificates, CSRs, install CA-issued certificates, and configure mTLS
Config Pack	Export device certificates, import/export encrypted configuration packages, view the configuration package list, and distribute

	secure configurations between devices

Task 1: Generate and use SSH keys to connect to a remote server

This task demonstrates how to generate SSH keys from scratch, deploy them to a remote Linux server (such as Jetson AGX Orin, workstation, etc.), and achieve passwordless login.

2.1 Generate SSH Key

Operating Steps

1. **Enter the Key Management Area** : Find the "Key Management" block at the top of the security page
2. **Click the Generate button** : Click the **"Generate New Key"** button
3. **Fill out the generation form**: Fill in the following information in the pop-up "Generate New Key" window:

Detailed Explanation of Form Fields

Field	Required	Description	Fill in Example
Key ID	is	The Unique Device Identifier of the key, used for subsequent reference. Naming rules: letters, numbers, underscores, and must be unique	`default`、`agx_key`、`lpmu_prod`

Key Type	is	Select encryption algorithm and key length	See "Key Type Selection" below
Remarks	No	Description of the key's purpose for easy identification in the future	`Used to connect to AGX Orin`, `Test Environment Key`
Alias	No	Friendly name displayed for the alternative key ID, recommended to fill in when "Hide Key" is enabled	`AGX Production Environment`, `Master Server Key`
Allows exporting private keys	No	After checking, you can export the private key for backup or migration. **Security Risk** : Private key leakage may lead to unauthorized access	Check only when backup is needed
Hide Key ID	No	When checked, low-privilege users cannot see the real key ID, only the alias or mask is displayed, which is suitable for multi-user environments	Check when multiple people share the device

Key Type Selection

Type	Key Length	Generation Time	Compatibility	Recommended Scenario
RSA 2048-bit	2048 bits	5-10 seconds	Best Compatibility	**Recommended**: For daily use
RSA 4096-bit	4096 bits	30-60 seconds	Best Compatibility	High-security scenario, slower

				generation
ECDSA P-256	256-bit	3-5 seconds	General	**Note**: SSH public key authentication is not currently supported
ECDSA P-384	384 bits	3-5 seconds	General	**Note**: SSH public key authentication is not currently supported

Important Notice: The current version of ECDSA keys does not support SSH public key authentication for the time being. Please select **RSA 2048-bit** or **RSA 4096-bit**.

Generation Process

4. **Submit the form**: After filling it out, click the 「**Generate**」 button
5. **Waiting for generation**:
   - RSA 2048: Approximately 5-10 seconds
   - RSA 4096: Approximately 30-60 seconds (Please be patient and do not close the page)
   - The pop-up window will display the generation progress
6. **Confirmation Successful** :
   - The pop-up window displays "Key generation successful"
   - After closing the pop-up window, the new key appears in the "Key Management" table
7. **View Key Information** : The table displays ID, Type, Remarks, Creation Time, and Whether it is Exportable

Precautions

- **Key ID Uniqueness** : If the entered ID already exists, an error will be prompted, and you need to replace it with another ID

- **Non-interruptible**: Do not refresh the page or close the browser during the generation process
- **Permanent Storage** : After key generation, it is securely stored in the device's NVS (Non-Volatile Storage) and will not be lost upon restart
- **Storage Limit** : NVS can store up to approximately 8 RSA-4096 keys or 32 ECDSA keys

---

2.2 Deploy the public key to the remote server

After generating the key, you need to deploy the **public key** to the remote Linux server to enable passwordless login.

Prerequisites

- has generated at least one key in "Key Management"
- Know the IP address or hostname of the remote server
- Know the username and password of the remote server (for initial deployment)
- The device is network-connected to the remote server

Operating Steps

1. **Select Key**: Locate the key row to be deployed in the "Key Management" table
2. **Click the Deployment button** : Click the **"Deploy"** button (blue) on the far right of this key row
3. **Fill out the deployment form**: In the pop-up window titled "Deploy Public Key to Remote Server", fill in the following:

Deploy form fields

Field	Required	Description	Fill in Example
Target Host	is	IP address or hostname of the remote server	`192.168.55.100`、`agx.local`、`10.10.99.98`
Username	is	Login username for the remote server	`root`、`nvidia`、`ubuntu`
Port	No	SSH service port, default 22. If the server uses a non-standard port, it needs to be modified	`22` (default), `2222`, `22022`
Authentication Password	is	The password used for authentication during the initial deployment. After successful deployment, subsequent connections do not require a password.	Your server login password

4. **Start Deployment** : Click **the "Start Deployment"** button
5. **View Progress** : The pop-up window will display the output of the deployment process, including:
   - Connect to Server
   - Verify the host key (the first connection will prompt you to trust the fingerprint, see "Task 3" for details)
   - Upload the public key to `~/.ssh/authorized_keys`
   - Verify the deployment results
6. **Confirmation Successful** :
   - Displays "Public key deployment successful"
   - Displays "Connection verification successful"
   - This host is automatically added to the "Deployed Hosts" list

Deployment Principle

The deployment process will perform the following operations:

1. Connect to the remote server via SSH using the provided password
2. Read the public key content of the key (OpenSSH format)
3. Append the public key to the end of the `~/.ssh/authorized_keys` file on the remote server
4. Verify if the deployment is successful (try to connect using the key)

Technical details: Similar to the `ssh-copy-id` command in Linux.

Common Error Handling

Error Prompt	Possible Reasons	Solution
Connection timed out	Network connectivity issues, firewall blocking, server SSH not enabled	Check network connectivity (ping), confirm SSH service is running (`systemctl status sshd`), and check firewall rules
Password is incorrect	Password input error, username incorrect	Reconfirm your username and password, paying attention to case sensitivity
Permission Denied	SSH configuration does not allow password login	Enable `PasswordAuthentication yes` in the server `/etc/ssh/sshd_config`
Host fingerprint verification failed	Trust the host when connecting for the first time	See the "Task 3: Manage Known Host Fingerprints" section for details
authorized_keys permission error	Remote `~/.ssh` directory permissions are incorrect	Ensure `~/.ssh` permissions are 700, `authorized_keys` permissions are 600

2.3 Use Key for Password-Free Login

After successful deployment, you can use the key to connect to the remote server without having to enter the password again.

Test the connection in WebUI

1. **Enter the deployed host area** : Scroll to the "Deployed Hosts" table
2. **Locate the target host** : Find the host just deployed in the list
3. **Click the Test button** : Click the **"Test"** button in this host row
4. **View Results**:
   - **Success** : Displays "Connection Successful" and the remote username
   - **Failure**: Display error messages (network issues, key mismatch, server configuration, etc.)

Connect using a key in the terminal

If you have **root privileges** to access the terminal page, or access the device's CLI through other means (such as serial port), you can use the following command:

代码块

```
# 启动交互式 Shell
ssh --host 192.168.1.100 --user nvidia --keyid agx_key --shell

# 执行单条命令
ssh --host 192.168.1.100 --user nvidia --keyid agx_key --exec "nvidia-smi"

# 仅测试连接
ssh --test --host 192.168.1.100 --user nvidia --keyid agx_key
```

Parameter Description:

- `--host` : Remote server address
- `--user` : Login Username
- `--keyid` : Use the key ID from the secure storage (consistent with the key ID at deployment)
- `--shell` : Start an interactive shell
- `--exec` : Exit after executing a single command

Troubleshooting Connection Failures

If password-free login fails, troubleshoot in the following order:

1. **Check if the key is correctly deployed** :
   - Check if the host exists in the "Deployed Hosts" list
   - Confirm that the key ID used is consistent with that at the time of deployment

2. **Check network connectivity** :
   - Ensure that the device can reach the server network
   - Try to ping the server IP

3. **Check server configuration** :
   - Confirm that the SSH service is running
   - check `/etc/ssh/sshd_config` - `PubkeyAuthentication yes`
   - Confirm `~/.ssh/authorized_keys` file exists and contains the public key

4. **View detailed logs**: Use the `--verbose` parameter in the terminal to view detailed connection logs

---

2.4 Revoke the deployed public key

When access to a server is no longer required, or when there is suspicion of key leakage, the deployed public key should be revoked in a timely manner.

When to Revoke

- No longer need to access this server
- The key may have been compromised and needs to be replaced
- Server user change, old keys need to be cleaned up
- Regular security reviews to clean up expired access permissions

Revocation Method 1: Revoke from Key Management

Applicable Scenarios: Know specifically which key and which server to revoke

1. **Select Key**: Find the key to be revoked in the "Key Management" table
2. **Click the Undo button**: Click the 「**Undo**」 button (orange)
3. **Fill out the revocation form**:

Field	Required	Description
Target Host	is	Server address for revoking the public key
Username	is	Server Login Username
Port	No	SSH port, default 22
Authentication Password	is	Server password (password authentication is required because the key may have expired)

4. **Confirm Revocation**: Click the 「**Revoke Public Key**」 button
5. **View Results**:
    - Shows how many lines of public keys were deleted from `authorized_keys`
    - Local "Deployed Host" records are automatically deleted

Revocation Method 2: Revoke from deployed hosts

Applicable Scenario: Quickly revoke in the list of "Deployed Hosts"

1. **Enter the deployed host area** : Scroll to the "Deployed Hosts" table
2. **Click the Undo button** : Click the **"Undo"** button (red)
3. **Enter Password** : Enter the server password in the pop-up revocation confirmation window
4. **Confirm Revocation** : Click the **"Revoke and Remove"** button
5. **Waiting for completion**: The system will:
    - Connect to the remote server
    - Remove the corresponding public key from `authorized_keys`
    - Delete local records of "deployed hosts"

Remove only local records (do not revoke remote public keys)

Applicable Scenarios:

- The server is disabled or inaccessible
- The public key has been manually deleted on the server side
- Only need to clean up local records and retain the remote public key

Operating Steps :

1. Find the target host in the "Deployed Hosts" table
2. Click the 「**Remove**」 button (gray)
3. Confirm Removal

4. Local records are deleted, and the `authorized_keys` **on the remote server will not** be modified

Differences between Undo and Remove

Operation	Remote Public Key	Local Record	Password required	Applicable Scenarios
Undo	Delete	Delete	Needs	Completely revoke access rights
Remove	Reserved	Delete	Not required	Clean local records, retain remote access

---

2.5 Export Key (Backup or Migration)

Export Public Key

Usage:

- Manually added to other servers' `authorized_keys`
- Share with other users or devices
- Backup public key content

Operating Steps :

1. Find the target key in the "Key Management" table
2. Click **the "Public Key"** button
3. In the pop-up "Public Key Export" window:

- View the content of the public key in OpenSSH format (e.g., `ssh-rsa AAAAB3NzaC1yc2E...`)
- Click 「**Copy to Clipboard**」 to copy the content
- or click **"Download File"** to save as `.pub` file

4. **Use public key**:
   - Paste it into the `~/.ssh/authorized_keys` file on the remote server
   - or send it to other administrators via email or chat tools

Export Private Key

Important Security Warning: Private keys are highly sensitive security credentials. Once compromised, anyone can use them to access deployed servers. Export private keys only in the following scenarios:

- Key backup (stored on a secure offline medium)
- Key migration to other devices
- Use this key on other SSH clients (such as desktop computers)

Prerequisites:

- Key generation was checked **"Allow export of private key"**
- If not checked, the "Private Key" button for this key will be grayed out and unavailable

Operation Steps:

1. Find the exportable key in the "Key Management" table
2. Click the **"Private Key"** button
3. **Read the security warning**: The system will pop up a warning dialog box, prompting the security risk of the private key
4. **Confirm to continue**: Click "OK"
5. In the "Private Key Export" pop-up window:
   - View the content of the private key in PEM format (`-----BEGIN RSA PRIVATE KEY-----` ...)

- Click 「**Copy to Clipboard**」 to copy the content
- or click **"Download File"** to save as `.pem` file

Suggestions for Secure Handling of Private Keys

After exporting the private key, be sure to follow the following security practices:

1. **Secure Transmission**:
   - Do not send private keys via email
   - Do not paste your private key in chat software
   - Do not upload to cloud storage (unless encrypted)
   - Use encrypted USB flash drives and external hard drives
   - Locally encrypted before transmission

2. **Secure Storage** :
   - Stored on an encrypted disk or in a safe
   - Set file permissions to 600 (only the owner can read and write)
   - Use a password manager for storage (e.g., 1Password, Bitwarden)

3. **Use post-processing**:
   - Delete the file immediately after temporary use
   - Clear Clipboard History
   - Do not use private keys on public computers

4. **Regular Review**:
   - Check for unauthorized use of public keys
   - Immediately revoke and regenerate upon suspicion of leakage

---

2.6 Delete keys that are no longer needed

Checklist before deletion

Before deleting the key, please confirm:

- [ ] This key has not been deployed to critical servers, or has already been revoked.
- [ ] Private key backup has been exported (if needed)
- [ ] Understand that deletion cannot be undone (unless there is a backup)
- [ ] Check the list of "Deployed Hosts" to confirm that no hosts are using this key

Operating Steps

1. **Select Key**: Find the key to be deleted in the "Key Management" table
2. **Click the Delete button**: Click the 「**Delete**」 button (red)
3. **Confirm deletion** : The system will pop up a confirmation dialog box, prompting that it cannot be restored after deletion
4. **Final Confirmation**: Click "OK" to execute deletion
5. **Verification Result**: Key disappeared from the list

Consequences of deletion

- The key is permanently deleted from NVS, freeing up storage space
- Deployed host records**remain**(manual revocation or removal required)
- Unable to connect to the deployed server using this key (unless regenerated and redeployed)
- Cannot be recovered (unless a private key backup was previously exported)

Cleanup work after deleting the key

1. **Clean up deployed hosts** :
    - Enter the list of "Deployed Hosts"
    - Find the host using this key
    - Click "Remove" to delete local records
    - (Optional) If the server is still accessible, revoke the public key first and then delete the key.

2. **Notify relevant personnel** (multi-user environment):
    - Inform other administrators that the key has been deleted
    - Update access documents and key inventory

Task 2: Manage deployed hosts

The "Deployed Hosts" section records which remote servers have had the device's public key deployed, facilitating your unified management and quick testing of connections.

3.0 Import Host Configuration (Optional)

If you have deployed the public key to a remote server via the terminal CLI (such as `ssh --copyid`) or manually, these hosts will not automatically appear in the "Deployed Hosts" list. You can **import host configurations** to import SSH host configurations (.tscfg configuration packages) from other devices or backups into this device, making the list consistent with the actual situation and facilitating unified testing, revocation, or removal in the WebUI.

Operation Steps:

1. **Enter the Deployed Hosts Area** : Find the "Deployed Hosts" block on the security page, with the **"Import Hosts"** button on the right
2. **Click Import Host**: Click the **「Import Host」** button to open the 「Import SSH Host Configuration」 pop-up window

3. **Select File**: Select a local `.tscfg` configuration package file (usually generated by the "Export Host Configuration" of other devices or the configuration package system)
4. **Preview and Confirm**: The system will verify the signature of the configuration package and display the configuration ID, signer, and remarks; if the configuration already exists, it will prompt whether to overwrite
5. **Optional**: Check "Overwrite existing configuration" to overwrite the existing configuration with the same ID
6. **Confirm Import**: Click 「**Confirm Import**」, and after the import is successful, refresh the page to see the corresponding record in the 「Deployed Hosts」 list

Instructions: The imported configuration package must be in a valid, signed SSH host configuration format; it is only used to synchronize the list of <Deployed Hosts> and will not modify the `authorized_keys` on the remote server.

3.0.1 Export Host Configuration (Single Item)

In the "Deployed Hosts" table, each row of hosts has **an "Export"** button, which can export the configuration of that host as an encrypted `.tscfg` file for backup or import to another device.

Operation Steps:

1. **Select Host**: Find the host to be exported in the "Deployed Hosts" table
2. **Click Export** : Click the **"Export"** button (blue) in this row to open the "Export SSH Host Configuration" pop-up window
3. **Target device certificate (optional)**:
   - **Leave blank**: Encrypt using the local machine's certificate (self-encryption), the exported.tscfg can only be decrypted on the local machine or devices with the local machine's certificate, suitable for backup
   - **Paste the target device certificate PEM** : If you want to send this host configuration to another device, you can paste the device certificate exported by the other party in the "Configuration Package" section. The exported.tscfg can only be decrypted and imported by that device

4. **Confirm Export** : Click **"Export"** , the system generates a.tscfg file and prompts for the download or save path

Typical Uses: Back up the configuration of a single host, synchronize the configuration of the same host among multiple TianshanOS devices, and cooperate with "Import Host" to complete device migration.

3.1 View the list of deployed hosts

List Field Description

Field	Description	Example
Host ID	Automatically generated unique identifier, format: `Key ID@Host Address`	`agx_key@192.168.55.100` 、 `default@agx.local`
Address	IP address or hostname of the remote server	`192.168.55.100` 、 `10.10.99.98`
Port	SSH Service Port	`22` 、 `2222`
Username	Remote Login User	`nvidia` 、 `root` 、 `ubuntu`
Deployment Key	Key ID used (displayed as a badge)	`agx_key` 、 `default`
Operation	Available buttons per row	Test, Export, Undo, Remove (see each subsection below)

When will the list be updated?

- Automatically added after successful deployment via the WebUI "Deployment" feature

- Automatically deleted after being undone via the WebUI "Undo" function
- Delete after manually clicking "Remove"
- **Note**: Hosts added via the terminal CLI or manually to `authorized_keys` will **not** automatically appear in this list; you can use the "Import Hosts" feature to import a.tscfg configuration package to synchronize the list

Empty List Handling

If the list shows "No deployed hosts yet":

- Indicates that you have not deployed any public keys via WebUI yet
- Please deploy the public key according to the steps in "Task 1 2.2"
- After successful deployment, the host will automatically appear in this list

Relationship with the Instruction Page

The data source of the "Select Host" dropdown list in the Instruction page (requires root privileges) is the "Deployed Hosts" on this security page. When exporting instruction configurations on the Instruction page, you can choose to **export the dependent host configurations simultaneously** , which facilitates importing the required hosts together when importing instructions on another device, eliminating the need to separately import hosts on the security page.

3.2 Test SSH Connection

Quickly verify the connection status of a deployed host.

Operating Steps

1. **Select a host**: Find the host to be tested in the "Deployed Hosts" table

2. **Click Test button** : Click **"Test"** button
3. **Waiting for testing**: The system will perform the following operations:
   - Connect to the server using the deployed key
   - Execute simple commands (such as `whoami`) to verify the connection
   - Check the return result
4. **View Results** : Display test results in a pop-up window or result area

Interpretation of Test Results

Successful Example :

```
代码块
SSH connection test successful
Host: 192.168.55.100:22
User: nvidia
Command: whoami
Output: nvidia
```

Failure Example:

```
代码块
SSH connection test failed
Error: Connection timeout
Suggestion: Check network connectivity and firewall settings
```

Failure handling suggestions

Error Type	Processing Method
Connection timed out	Check network, confirm server is online, check firewall

Key authentication failed	Public keys may be deleted, redeployed, or checked `authorized_keys`
Host fingerprint mismatch	The server may be reinstalled, update the host fingerprint (see "Task 3 4.4")
Permission Denied	Check the SSH configuration of the server and the permissions of the public key file

---

3.3 Revoke Public Key

See "Task 1 2.4 Revocation Method 2: Revoke from Deployed Hosts" for details.

3.4 Remove Host Record

Differences from Undo

Operation	Local Record	Remote Public Key	Applicable Scenarios
Undo	Delete	Delete	Access rights need to be completely revoked
Remove	Delete	Reserved	Server has been deactivated, public key has been manually

			deleted, only local data has been cleared

Applicable Scenarios

- The server has been deactivated or destroyed
- The public key has been manually deleted on the server side
- Host record error, needs to be cleaned up
- Only need to clean up the local list, while the remote public key remains

Operating Steps

1. **Select Host**: Find the target host in the "Deployed Hosts" table
2. **Click the Remove button**: Click the 「**Remove**」 button (gray)
3. **Confirm Removal**: The system pops up a confirmation dialog box
4. **Final Confirmation**: Click "OK"
5. **Verification Result**: Host disappeared from the list

Note: The removal operation **does not** connect to the remote server and only deletes records in the local database. If you need to delete the remote public key, please use the "Revoke" function.

Task 3: Manage known host fingerprints (to prevent man-in-the-middle attacks)

The "Known Host Fingerprint" feature prevents connections to forged servers (man-in-the-middle attacks) by recording the host key fingerprint of the SSH server.

4.1 What is a known host fingerprint?

TOFU（Trust On First Use）system

Working Principle:

1. **When connecting to a SSH server for the first time**, the server will send its host key (public key)
2. The Client calculates the **SHA256 hash value** (fingerprint)
3. Prompt the user whether to trust this fingerprint
4. After user confirmation, the fingerprint is saved to the NVS of the device
5. **When subsequently connecting** , automatically compare the server fingerprint with the saved fingerprint
6. If it matches, pass silently; if it does not match, issue **a security warning**

Functions of Fingerprints

- **Prevent man-in-the-middle attacks**: Ensure that the connection is to a genuine server, not a server forged by an attacker
- **Server Authentication** : Verify that the server's key has not been tampered with
- **Connection Records**: Track which servers have been connected to previously

Fingerprint Format

The fingerprint is the SHA256 hash value of the server host key, with the format as follows:

代码块

```
SHA256:xYzAbC123456789aBcDeF/gHiJkLmNoPqRsTuVwXyZ01=
```

4.2 View the list of known host fingerprints

List Field Description

Field	Description	Example
Host	Address of the SSH server	`192.168.1.100`、`agx.local`
Port	SSH Service Port	`22`、`2222`
Key Type	Host key algorithm used by the server	`ssh-rsa`、`ssh-ed25519`、`ecdsa-sha2-nistp256`
Fingerprint (SHA256)	Hash value of the server host key	`SHA256:xYzAbC... (partially shown)`
Add Time	Time of first trusting this server	`2026-01-30 14:30:00`

View full fingerprint

The fingerprint in the list may be truncated for display. To view the full fingerprint:

1. **Click the View button** : Click the **"View"** button in the target host row
2. **View Details**: The system displays the **complete SHA256 fingerprint** of the host, as well as the host, port, and key type, in a pop-up window, facilitating a verbatim comparison with the output of `ssh-keygen -lf` on the server side
3. **Comparison and Verification** (Optional): In high-security scenarios, compare the fingerprint in the pop-up window with the official fingerprint on the server, and trust it only after confirming their consistency

Obtain the server-side fingerprint (run on the server):

代码块

```
# ECDSA 密钥
ssh-keygen -lf /etc/ssh/ssh_host_ecdsa_key.pub

# ED25519 密钥
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

# RSA 密钥
ssh-keygen -lf /etc/ssh/ssh_host_rsa_key.pub
```

Output Example:

代码块

```
256 SHA256:xYzAbC123456789aBcDeF/gHiJkLmNoPqRsTuVwXyZ01= root@server (ECDSA)
```

4.3 First Connect to a New Server

Connection Process

When you first connect to a new SSH server (deploying a public key or using key-based connection), the system will prompt you to trust the host:

Step 1: Initiate Connection

- Perform operations to deploy public keys or SSH connections

Step 2: System Prompt

代码块

```
┌──────────────────────────────────────────────────────────────────────┐
│  NEW HOST - First time connecting to this server                     │
└──────────────────────────────────────────────────────────────────────┘

Host: 192.168.1.100:22
Key Type: ecdsa-sha2-nistp256
Fingerprint: SHA256:xYzAbC123456789aBcDeF/gHiJkLmNoPqRsTuVwXyZ01=

This is the first time connecting to this host.
Trust this host and continue? (yes/no):
```

Step 3: User Decision

You have two options:

Input	Result	Applicable Scenarios
yes	Trust this server, save the fingerprint to NVS, and continue connecting	Confirm it is the target server
no	Abort connection without saving fingerprint	Unable to determine the server identity, or an incorrect IP was entered

Step 4: After Trust

- The fingerprint is saved to the device's "Known Host Fingerprints" list
- This connection will continue to execute
- Automatically authenticate when connecting to this server in the future, no need to confirm again

How to verify fingerprint authenticity (high-security scenarios)

In enterprise production environments or scenarios with high security requirements, fingerprints should be verified before the first connection:

1. **Obtain official fingerprint** :
   - Contact the server administrator and request the official host key fingerprint
   - Or directly log in to the server and run `ssh-keygen -lf /etc/ssh/ssh_host_*_key.pub`

2. **Compare Fingerprint**:
   - Compare the fingerprint displayed by the system with the official fingerprint character by character
   - Ensure complete consistency (including key type)

3. **Trust after Confirmation**:
   - Fingerprint match: Input `yes` Trust
   - Fingerprint mismatch: Input `no` to abort and troubleshoot the issue

---

4.4 Host Fingerprint Mismatch Warning

Trigger Scenario

When you connect to a server with a saved fingerprint, if the fingerprint returned by the server does not match the saved one, the system will display **a security warning** .

Possible Reasons :

Reason	Risk Level	Description
	High Risk	

Man-in-the-Middle Attack (MITM)		The attacker intercepts and forges the server in the network
Server System Reinstallation	Legal	SSH host key is regenerated after reinstalling Linux
Server Key Replacement	Legal	The administrator manually regenerated the SSH host key
IP Address Reassignment	Legal	This IP is now assigned to a different server

Warning Popup Content

The system will pop up a red warning window:

代码块

```
Security warning: Host fingerprint mismatch
The host key has been changed! This may indicate:
• Man-in-the-Middle attack
• Server reinstallation or key regeneration
• IP address assigned to a different server

Host: 192.168.1.100:22
Stored fingerprint: SHA256:OldFingerprint123...
Current fingerprint: SHA256:NewFingerprint456...

Suggestion: If you confirm that the server has been reinstalled or the key has been updated, you can click "Update Host Key" to remove the old record and reconnect to trust the new key.
```

Correct handling steps

Step 1: Stop the connection

- Do not blindly click "Update Host Key"

- Verify the situation first

Step 2: Verify the change

Verify by selecting one of the following methods:

1. **Contact the server administrator**:
   - Ask if the system has been recently reinstalled
   - Ask if the SSH host key has been regenerated
   - Request for a new official fingerprint

2. **Check server logs** (if accessible):
   - Check the system installation date: `ls -l /var/log/installer/`
   - Check the generation time of SSH keys: `ls -l /etc/ssh/ssh_host_*_key`

3. **Physical Verification** (if the server is accessible):
   - Run directly on the server `ssh-keygen -lf /etc/ssh/ssh_host_*_key.pub`
   - Compare the "Current Fingerprint" in the pop-up window

Step 3: Process according to the verification results

Case A: Confirmed as a legitimate change (server reinstallation, key replacement)

1. Click the **"Update Host Key"** button in the warning pop-up window
2. Old fingerprint has been deleted
3. Re-execute the connection operation
4. The system will prompt you to trust the new fingerprint again (follow the first connection process in 4.3)

Scenario B: Unable to confirm or suspect an attack

1. Click **「Cancel」** button
2. **Do not**continue connecting to this server

3. Immediately conduct a thorough investigation:
    - Check the network environment (whether it is on an insecure public WiFi)
    - Contact the network administrator to check for ARP spoofing and DNS hijacking
    - Try connecting using other devices or networks and compare fingerprints

Consequences of incorrect operation

- **Blindly trusting new fingerprints** : If it is indeed a man-in-the-middle attack, your password and data will be stolen by the attacker
- **Ignore warnings** : Repeatedly ignoring warnings may lead to the formation of a habit and reduce safety awareness

Security Principle: It is better to interrupt the connection for troubleshooting than to take the risk of trusting an unverified fingerprint.

4.5 Delete Host Fingerprint

Applicable Scenarios

- The server has been permanently decommissioned or destroyed
- The server has been reinstalled, and you need to clear the old fingerprint and re-trust it.
- The IP address has been assigned to another server, and the old record is invalid
- Regularly clean up server records that are no longer in use

Operating Steps

1. **Select Host** : Find the host to be deleted in the "Known Host Fingerprint" table
2. **Click the Delete button**: Click the 「**Delete**」 button

3. **Confirm deletion** : The system pops up a confirmation dialog box
4. **Final Confirmation**: Click "OK"
5. **Verification Result**: Host disappeared from the list

Consequences of deletion

- Fingerprint records are permanently deleted from NVS
- When connecting to this server next time, you will be prompted to trust the fingerprint again (following the first connection process)
- If the server is reinstalled, **the old fingerprint must be deleted first** , otherwise it will continuously prompt that the fingerprint does not match

Batch Delete

If you need to clear all known host fingerprints (such as resetting a test environment), you can execute the following command via the terminal CLI:

代码块

```
hosts --clear --yes
```

Warning: This operation will delete all known host fingerprints. Use with caution.

4.6 Handling when the known host list is empty

Possible Reasons

If the "Known Host Fingerprint" area always displays "No known hosts" when opening the security page, it may be because:

1. **First Use**: You have not connected to any SSH servers yet
2. **Module not initialized**: Known host module not properly initialized during security service startup (known bug)

Solution

Method 1: Trigger Initialization

- Perform an SSH connection operation (such as deploying a public key)
- After trusting the host, refresh the page
- The list should display records

Method 2: Check Initialization

- If Method 1 is ineffective, refer to the section "Known Host List Initially Empty" in the project document <Troubleshooting>
- May need to check if the security service initialization called `ts_known_hosts_init()`

Task 4: Manage HTTPS Certificates (mTLS Mutual Authentication)

The "HTTPS Certificate" block is used to configure the device's mTLS (Mutual TLS Authentication) certificate, enabling the device to act as an HTTPS server and perform mutual authentication with the Client.

5.1 Overview of HTTPS Certificates

What is mTLS?

mTLS (Mutual TLS) is mutual TLS authentication:

- **Traditional HTTPS**: Client verifies server certificate (one-way)
- **mTLS**: Server and Client mutually verify certificates (two-way)

Advantages:

- Two-way identity authentication to prevent unauthorized Client access
- Prevent man-in-the-middle attacks
- Certificate-based access control (no password required)

Certificate Architecture

代码块

```
┌─────────────────┐
│   Root CA   │ Trust Anchor (offline storage)
│   20-year   │
│   validity  │
└─────────────────┘
        │ Issued
        ▼
┌─────────────────┐
│Intermediate │ Online certificate issuance
│     CA      │ (e.g., step-ca server)
│   10-year   │
│   validity  │
└─────────────────┘
        │ Issued
        ▼
┌─────────────────┐
│   Device    │
│ certificate │ This device's identity proof
│    1 year   │
│   validity  │ (stored in NVS)
│             │
└─────────────────┘
```

Principle of Private Key Not Leaving the Device

Security Design:

- Device private keys are generated locally and never exported
- Export only CSR (Certificate Signing Request, without private key)
- CA uses CSR to issue certificates, and CA cannot obtain the device's private key
- The private key is always stored in the device's NVS and remains even after firmware upgrades

Compare SSH Keys :

- SSH keys can choose "Allow private key export"
- HTTPS Certificate Private Key**Must Never Be Exported**(More Strict)

5.2 Check Certificate Status

Status Card Information

At the top of the "HTTPS Certificate" section, there is a status card that displays the certificate status in real time.

Status and Meaning

Status Text	Description
Loading...	is retrieving the certificate status
Key not generated	Key pair has not been generated yet, please generate it first

Key generated, certificate not installed	The key pair has been generated, but the CA-issued certificate has not been installed
Activated	The certificate has been installed and is valid
is about to expire	The certificate will expire within 30 days, it is recommended to renew it
Expired	The certificate has expired, and mTLS functionality is unavailable

Certificate Details (When Installed)

When the certificate status is "Activated" or "Expiring Soon", the card will expand to display detailed information:

Field	Description	Example
Subject CN	Unique Identifier (Common Name) of the Device	`TIANSHAN-RM01-0001`
Issuer	Name of the CA that issued this certificate	`TianShanOS Intermediate CA`
Effective Time	Start effective time of the certificate	`2026-01-27 00:00:00`
Expiration Time	End validity time of the certificate	`2027-01-27 00:00:00`
Serial Number	Unique serial number (hexadecimal) of the certificate	`01:23:45:67:89:AB`
Effective Status	Is it currently within the validity period?	Valid / Expired

Remaining Days	How many days are left until expiration?	`365 days` , `28 days` (<30 days shows a warning)

Status Card Display Rules:

- **Remaining Days**: Displayed only when the certificate is installed and within its validity period; not displayed when the certificate is not installed or has expired.
- **"Expiring Soon" Badge/Status** : When the certificate is valid but less than 30 days from expiration, the status text will change to "Expiring Soon", and a badge reminder may be displayed. It is recommended to renew it as soon as possible.
- The detailed information of the expanded card (subject CN, issuer, effective/expiration time, serial number, validity status, remaining days) is only displayed when the status is "activated" or "expiring soon".

Button Status Description

There are 6 operation buttons below the HTTPS certificate block, which are enabled or disabled based on the current state:

Button	Key not generated	Key generated	Installed Certificate	Description
Generate Key Pair	Available	Available	Available	Can be generated at any time (will overwrite the old key)
Generate CSR	Disable	Available	Available	A key pair is required first
Install Certificate	Disable	Available	Available	A key pair is required first
Install CA	Disable	Available	Available	A key pair is required first
View Certificate	Disable	Disable	Available	Requires installed certificate

Delete Voucher	Disable	Available	Available	Can be deleted when there is a key or certificate

---

5.3 Complete Certificate Application Process

Configuring the mTLS certificate of a device from scratch, the complete process includes 5 steps. The button and pop-up window names mentioned below (such as "Generate Key Pair", "Certificate Signing Request (CSR)", "Install Device Certificate", "Install CA Certificate Chain", "View Device Certificate") are all consistent with those in the Security Page WebUI, facilitating corresponding operations on the interface.

Process Overview

代码块

```
Step 1: Generate key pair (device end)
   ↓
Step 2: Generate CSR (device end)
   ↓
Step 3: Submit for CA issuance (external CA server)
   ↓
Step 4: Install device certificate (device end)
   ↓
Step 5: Install CA certificate chain (device end)
```

5.3.1 Generate Key Pair

Objective: Generate an ECDSA P-256 key pair on the device for subsequent CSR generation and TLS handshake.

Operation Steps:

1. **Check Status**:
   - Check the certificate status card to confirm the current status
   - If "Key Generated" is displayed, skip this step (unless you need to regenerate it)

2. **Click the Generate button** : Click the **"Generate Key Pair"** button

3. **Read Instructions**: The popped-up "Generate HTTPS Key Pair" dialog box shows:

   代码块

   ```
   Generate an ECDSA P-256 key pair for the device for mTLS authentication.
   ```

4. **Attention Override Warning** :
   - If a key pair already exists, a yellow warning will be displayed:

     代码块

     ```
     Note: A key pair already exists. Continuing will overwrite the existing key!
     ```

   - **Caution**: After overwriting the key, the CSR and certificate generated by the old key will become invalid
   - Confirm to overwrite before proceeding

5. **Start Generation** : Click **the "Generate"** button

6. **Waiting for completion**:
   - Generating an ECDSA P-256 key takes approximately 3-5 seconds

- Pop-up window displays progress or result information

7. **Confirmation Successful** :
   - Displays "Key pair generated successfully"
   - Close Popup
   - The status card is updated to "Key generated, certificate not installed"
   - The buttons "Generate CSR", "Install Certificate", and "Install CA" become available

Technical Description

- **Key Type** : ECDSA P-256 (256-bit Elliptic Curve Key)
  - Advantages: Fast generation, high encryption strength, and small key size
  - Standard： NIST P-256 / secp256r1
- **Storage Location**: Device NVS (Non-Volatile Storage), data will not be lost upon restart
- **Usage**:
  - HTTPS Server Certificate (Device as Server)
  - mTLS Client Certificate (Device connects to other mTLS services as a client)
- **Security**: Private keys are never exported and are only used locally on the device

---

5.3.2 Generate CSR (Certificate Signing Request)

Objective: Generate a CSR (Certificate Signing Request) and submit it to a CA (Certificate Authority) for certificate issuance.

Prerequisites:

- Completed "5.3.1 Generate Key Pair"
- The "Generate CSR" button is enabled (blue and clickable)

Operation Steps:

1. **Click the Generate CSR button**: Click the 「**Generate CSR**」 button

2. **Fill in CSR Information**: Fill in the following in the "Certificate Signing Request (CSR)" pop-up window:

Field	Required	Description	Fill in Example
Device ID (CN)	No	Unique identifier (Common Name) of the device, leave blank to use the default configuration	`TIANSHAN-RM01-0001`、`DEVICE-AGX-001`
Organization (O)	No	Company or Organization Name (Organization)	`HiddenPeak Labs`、`My Company`
Organizational Unit (OU)	No	Organizational Unit (OU)	`Device`、`Production`、`IoT`

Field Description:

- **CN (Common Name)**: Most important, it is recommended to use a standardized naming convention, such as `prefix-model-serial number`
- **O and OU**: Optional, used for organizational information in certificates
- **Leave Blank**: If all fields are left blank, the system uses the default configuration (e.g., device MAC address as CN)

3. **Generate CSR**: Click the **"Generate CSR"** button

4. **Waiting for generation**: Approximately 2-3 seconds, the system will:
   - Generate CSR using the private key and the filled information
   - CSR format complies with X.509 standard (RFC 2986)

5. **View CSR**: After successful generation, the pop-up window will expand to display:

代码块

```
CSR content (to be copied to the CA server for issuance)
┌──────────────────────────────────────────────────────────┐
│ -----BEGIN CERTIFICATE REQUEST-----        │
│ MIIBVTCByQIBADBGMQswCQYDVQQGEwJVUzEWMBQG   │
│ A1UECgwNSGlkZGVuUGVhayBMYWJzMR8wHQYDVQ     │
│ ... (CSR content encoded in Base64) ...    │
│ -----END CERTIFICATE REQUEST-----          │
└──────────────────────────────────────────────────────────┘
```

6. **Copy CSR**: Click the 「**Copy to Clipboard**」 button to copy the CSR content

7. **Save CSR** (Optional):
   - Paste the CSR into a text editor
   - Save as `.csr` file (e.g., `device.csr`)
   - Facilitates subsequent submission to CA

Information Contained in CSR

- Device's public key (extracted from the key pair)
- Subject Information (CN, O, OU)
- Device signature (proving that the requester possesses the private key)
- **Does not contain**private keys (private keys never leave the device)

Next Step

Submit the copied CSR to your CA for issuance. For details, see "5.3.3 Submit to CA for Issuance".

5.3.3 Submit CA Issuance (External Operation)

Objective: Submit the CSR to the Certificate Authority (CA) to obtain the issued device certificate.

Note: This step is completed **externally** to the device and requires access to the CA server or contacting the PKI administrator.

Common CA Methods

Method 1: Internal enterprise CA (e.g., step-ca)

If your organization uses step-ca or other internal CAs:

1. **Log in to the CA server** (or use the step CLI)
2. **Submit CSR to Issue Certificate**:

   代码块

   ```
   # Save CSR as device.csr, then run
   step ca certificate --csr device.csr device.crt

   # or directly paste the CSR content
   cat > device.csr << 'EOF'
   -----BEGIN CERTIFICATE REQUEST-----
    (paste the CSR content)
   -----END CERTIFICATE REQUEST-----
   EOF

   step ca certificate --csr device.csr device.crt
   ```

3. **Obtain Certificate and CA Chain** :
   - `device.crt` : Device certificate (required for the next step)

- `ca-chain.pem` : CA certificate chain (downloaded from the CA server)

Method 2: Let's Encrypt (Public CA)

If the device has a public domain name, you can use Let's Encrypt:

1. Automatically issue using the ACME protocol
2. Domain ownership verification (HTTP-01 or DNS-01 verification) is required
3. Issue a free 90-day certificate

Note: Let's Encrypt is more suitable for public network services, and it is recommended to use an enterprise internal CA for internal network devices.

Method 3: Manual issuance (PKI administrator)

1. **Submit CSR**: Send the CSR content to the PKI administrator (email, ticket system)
2. **Waiting for issuance**: After administrator review, use the CA private key to issue the certificate
3. **Obtain Certificate** : Receive the issued device certificate (.crt or.pem file)

Waiting Time

- **Automated CA**: Seconds to Minutes
- ⏳ **Manual CA**: Hour to day level (depending on the approval process)

Retrieved File

After issuance is completed, you should obtain the following documents:

File	Required	Description	Format
Device Certificate	is	Device certificate issued by CA	`device.crt`, PEM format
CA Certificate Chain	is	Root CA + Intermediate CA Certificate	`ca-chain.pem`、PEM format

After preparing these two files, proceed to "5.3.4 Install Device Certificate".

5.3.4 Install Device Certificate

Objective: Install the device certificate issued by CA onto the device to enable the mTLS function.

Prerequisites:

- Key pair has been generated
- has obtained the issued device certificate (in PEM format) from the CA
- The certificate was issued using a CSR generated by the device key

Operation Steps:

1. **Open the certificate file**: Open the CA-issued certificate file on your computer (e.g., `device.crt`)

2. **Copy Certificate Content**: Select all and copy the certificate content, the format should be:

代码块

```
-----BEGIN CERTIFICATE-----
MIICxzCCAa+gAwIBAgIRAMxBxOxxxxxxxxxxxxxxxxxxxxx
 (Multiple line Base64 encoded content)
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxQ=
```

```
-----END CERTIFICATE-----
```

3. **Click the Install button** : On the security page, click **the <Install Certificate>** button

4. **Paste Certificate** : In the "Install Device Certificate" pop-up window:
   - Paste the certificate content into the "Certificate PEM" text box
   - Ensure the content is complete (including the `-----BEGIN/END-----` at the beginning and end)

5. **Submit Installation**: Click the **「Install」** button

6. **Waiting for verification**: The system will automatically verify:
   - Is the certificate format correct (PEM format)?
   - Whether the certificate matches the device private key (critical)
   - Is the certificate within the validity period?
   - Does the certificate subject information meet expectations?

7. **Confirmation Successful** :
   - Displays "Certificate installed successfully"
   - Show certificate details (CN, issuer, validity period, etc.)
   - The status card is updated to "Activated" or "Key Generated, CA Chain Not Installed"

Common Error Handling

Error Prompt	Reason	Solution
Certificate and private key do not match	The certificate was not issued using the CSR of the current device	① Regenerate the CSR and submit it to the CA for issuance ② or delete the certificate and start over

Certificate format error	The content is not in valid PEM format	Check if the copied content is complete and includes the BEGIN/END markers
The certificate has expired	The certificate issued by CA has expired	Contact the CA to reissue the certificate within the validity period
Failed to parse the certificate	Certificate content is corrupted or in an incorrect format	Reobtain the certificate file from the CA to ensure its content is complete

Principle of Verifying the Match between Certificate and Private Key

The system will perform the following verification:

1. Extract the public key from the certificate
2. Load the private key from the device NVS
3. Verify whether the public key and private key are a paired key pair
4. Only matching certificates can be successfully installed

Why it matters: Mismatched certificates cannot be used for TLS handshakes, which will lead to connection failures.

5.3.5 Install CA Certificate Chain

Objective: Install the CA certificate chain (Root CA + Intermediate CA) to enable the Client to verify the issuer of the device certificate.

Prerequisites:

- Device certificate installed (completed 5.3.4)
- has obtained the certificate chain file (PEM format) from the CA

Why is a CA chain needed?:

- The Client (browser, API client) needs to verify whether the device certificate is issued by a trusted CA
- CA chain provides a complete trust path: `Device Certificate ← Intermediate CA ← Root CA`
- If the CA chain is missing, the Client will prompt "Unable to verify certificate"

Operation Steps:

1. **Prepare CA chain file**:
   - File names such as `ca-chain.pem`, `root-ca.pem`, etc.
   - The content includes root CA and intermediate CA certificates (which can be concatenated in a single file)

2. **Open and copy the CA chain**: Select all and copy the content of the certificate chain, with the format:

   代码块

   ```
   -----BEGIN CERTIFICATE-----
    (Root CA certificate - Base64 content)
   -----END CERTIFICATE-----
   -----BEGIN CERTIFICATE-----
    (Intermediate CA certificate - Base64 content)
   -----END CERTIFICATE-----
   ```

Instructions :

- can contain multiple certificate blocks (root CA, one or more intermediate CAs)
- Order does not matter (the system will automatically recognize the root CA and intermediate CA)
- If there is only one CA (simplified environment), only one certificate block is needed

3. **Click the Install CA button**: Click the 「**Install CA**」 button

4. **Paste Certificate Chain** : In the "Install CA Certificate Chain" pop-up window:
   - Paste the CA chain content into the text box
   - Check that the content is complete (including all BEGIN/END tags)

5. **Submit Installation**: Click the 「**Install**」 button

6. **Waiting for completion**: The system verifies the certificate chain format and stores it in NVS

7. **Confirmation Successful** :
   - Displays "CA Chain Installation Successful"
   - Status card updated to "Activated" (if the certificate is valid)
   - mTLS configuration completed

CA Chain Example

Single-level CA (Simplified Environment):

代码块

```
-----BEGIN CERTIFICATE-----
 (self-signed root CA)
-----END CERTIFICATE-----
```

Two-level CA (Standard Enterprise Environment):

代码块

```
-----BEGIN CERTIFICATE-----
 (Root CA certificate)
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
 (Intermediate CA certificate)
-----END CERTIFICATE-----
```

Verification after installation

1. **Check Status Card** :
   - Status should be "Activated"
   - Valid status displays "Valid"
   - Remaining days should display the normal value

2. **Test mTLS Connection** (Requires Client Configuration):
   - Connect to the device using a Client that supports mTLS
   - Verify whether mutual authentication is successful

5.4 View Installed Certificates

Usage:

- View the complete PEM content of the device certificate
- Confirm certificate information (CN, validity period, etc.)
- Copy the certificate and share it with other systems (e.g., configure Client trust)

Operation Steps:

1. **Prerequisites** : The device certificate has been installed (the "View Certificate" button is in an available state)

2. **Click the View button** : Click **the "View Certificate"** button

3. **Waiting to load**: The pop-up window displays "Loading...", lasting approximately 1-2 seconds

4. **View Certificate Content**:
   - Display the complete PEM content of the device certificate
   - Scroll to view all content

5. **Copy Certificate** (Optional):
   - Click 「**Copy to Clipboard**」 to copy the certificate
   - Usage: Configure Client, Archive Backup

6. **Close the pop-up window**: Click the "Close" button

Interpretation of Certificate Content

The PEM content of the certificate is a Base64-encoded X.509 certificate, and tools can be used to parse and view detailed information:

Parse Certificate on Linux/macOS :

代码块

```
# Save the certificate as device.crt, then run
openssl x509 -in device.crt -text -noout
```

Output Example:

代码块

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 01:23:45:67:89:ab
        Signature Algorithm: ecdsa-with-SHA256
        Issuer: CN=TianShanOS Intermediate CA
        Validity
            Not Before: Jan 27 00:00:00 2026 GMT
            Not After : Jan 27 00:00:00 2027 GMT
        Subject: CN=TIANSHAN-RM01-0001, O=HiddenPeak Labs, OU=Device
        Subject Public Key Info:
```

```
                Public Key Algorithm: id-ecPublicKey
                    Public-Key: (256 bit)
```

5.5 Delete all PKI credentials (factory reset)

Warning: This operation will delete all HTTPS certificate-related data. Please confirm before proceeding.

Delete Content

Clicking "Delete Voucher" will delete all of the following content:

- Device Private Key (ECDSA P-256)
- Device Certificate
- CA Certificate Chain

When to Use

Scene	Description
Device Reconfiguration	Device ID or certificate system needs to be replaced
Certificate System Change	Company replaces CA or adjusts PKI architecture
Test Environment Reset	Clear the test data and retest the certificate process
Security Incident Handling	Suspect private key leakage, need to

	regenerate

Operating Steps

1. **Click the Delete button**: Click the 「**Delete Voucher**」 button (red)

2. **Read Warning**: The system pops up a confirmation dialog box:

   代码块

   ```
   Warning: Delete all PKI credentials
   This operation will delete:
   • Device private key
   • Device certificate
   • CA certificate chain

   After deletion:
   • mTLS functionality will be unavailable
   • Keys and certificates will need to be regenerated
   • This operation cannot be undone

   Are you sure you want to continue?
   ```

3. **Confirm Deletion**:
   - After confirming understanding of the consequences, click "OK"
   - If you're unsure, click "Cancel"

4. **Verification Result**:
   - Status card updated to "Key not generated"
   - All buttons are reset to their initial state
   - The HTTPS key row in the "Key Management" table has disappeared

Impact after deletion

- **HTTPS Service**:
  - may be downgraded to HTTP (if configuration allows)
  - or use a temporary self-signed certificate
  - mTLS mutual authentication failed

- **Client Connection**:
  - Clients using mTLS are unable to verify device identity
  - The browser may display a certificate error

- **Recovery Method**:
  - Re-execute "5.3 Complete Certificate Application Process"
  - Restart from Step 5.3.1

5.6 Certificate Renewal

When is renewal required?

- Certificate is about to expire (it is recommended to start renewal **30 days** in advance)
- Status card shows "About to expire" or remaining days < 30
- Renew periodically (e.g., annually) according to the company's PKI policy

Renewal Process

Method 1: Retain the key and only renew the certificate (recommended)

If the private key has not been compromised, you can retain the key pair and only update the certificate:

1. **Generate New CSR**:
    - Click "Generate CSR" (you can use the same CN)
    - Copy CSR Content

2. **Submit for CA Issuance**:
    - Submit the new CSR to the CA
    - Obtain newly issued certificates

3. **Install New Certificate** :
    - Click "Install Certificate"
    - Paste the content of the new certificate
    - The system will automatically overwrite the old certificate

4. **Verification Status**:
    - Check the status card, and the "Expiration Date" should be updated to the new date
    - "Remaining Days" should be reset to the validity period of the new certificate

Method 2: Completely regenerate (when the key is compromised)

If you suspect that the private key has been compromised, you need to regenerate all credentials:

1. **Delete old vouchers**: Click "Delete Voucher"
2. **Regenerate Key Pair** : Starting from "5.3.1"
3. **Reapply for Certificate** : Execute the complete 5.3 process

Renewal Notes

- The old certificate can continue to be used (until it expires) before renewal
- The new certificate will take effect immediately after installation, and the old certificate will automatically become invalid.

- CA chains usually do not need to be updated (unless the CA system changes)
- If the CA chain has also expired, it is necessary to update the CA chain simultaneously

Automated Renewal (Future Feature)

Note : The current version requires manual renewal. Future versions may support:

- Automatically detect that the certificate is about to expire
- Automatically generate CSR and submit it to ACME CA
- Automatically install the renewed certificate

Task 5: Configuration Package (Config Pack)

The "Configuration Package" block is used to securely distribute configurations between devices: encrypt and sign configuration files, then package them into `.tscfg` files, which can only be decrypted and used by target devices holding the corresponding device certificates. It is suitable for scenarios such as batch deployment and remote configuration distribution.

5.5.1 Configuration Package Function Overview

Function	Description	Permissions
Export Device Certificate	Export the HTTPS certificate (PEM) and fingerprint of this device for others to send encrypted configuration packages to this device	admin
Import configuration package	Upload or paste the.tscfg file, decrypt it after verifying the signature, and	admin

	save it to the device (encrypted storage)	
Export Configuration Package	Select a local file, fill in the target device certificate, generate an encrypted.tscfg package, and send it to the target device	Only Developer devices
View the list of configuration packages	View existing configuration package files, signers, official markings, and status by directory	admin

Status Card: The top of the block displays the current device type, certificate CN, certificate fingerprint, format version, etc.; if the mTLS certificate has not been configured, some functions may not be available.

5.5.2 Export Device Certificate

Usage: Send the certificate of this device to developers or systems that need to send encrypted configurations to you, so that the other party can paste your certificate when "exporting the configuration package" to generate a configuration package that can only be decrypted by this device.

Step :

1. Click **"Export Device Certificate"**
2. View certificate fingerprint (SHA256), certificate CN, and certificate PEM in the pop-up window
3. Click **「Copy to Clipboard」** or copy the PEM content manually, then send it to the other party via a secure channel

5.5.3 Import Configuration Package

Purpose : Receive the.tscfg configuration package issued by others or the system, verify the signature, decrypt it, and then save it to the device.

Step :

1. Click **「Import Configuration Package」**
2. Select a.tscfg file or paste JSON content
3. You can first click **"Verify Only"** to check the signature and content, then click **"Import"** to write to the device

Explanation: "Verify Only" only checks the signature and format of the configuration package,**without**writing the configuration to the device, and is suitable for confirming whether the source and content are trustworthy before formal import.

4. After successful import, the configuration package will be encrypted and stored on the device, and can be decrypted and used as needed via `config.pack.content` and other APIs

5.5.4 Export Configuration Package (Developer Devices Only)

Usage: Select a configuration file on this device, encrypt and sign it with the certificate of the target device, and generate a.tscfg file for the target device to import.

Step :

1. If the device is not of the Developer type, this button will prompt "Only Developer devices can export configuration packages".
2. Click **「Export Configuration Package」**, then browse and select the configuration files to be packaged (multiple selection allowed) in the pop-up window
3. Fill in the configuration name and description, and paste the**target device**exported certificate PEM
4. Click **「Generate Configuration Package」**, copy or download the generated.tscfg content, and then send it to the target device

5.5.5 View Configuration Package List

Usage: List existing configuration package files in the specified directory (e.g., `/sdcard/config`), and view the file name, size, signer, official status, state, etc.

Step : Click **"View Configuration Package List"** , enter or use the default directory path, click **"Refresh"** to view the list.

List Field Description:

- **Signer**: The certificate CN that signs the configuration package, used to verify the source
- **Official**: If the signature comes from an official certificate trusted by the system, it will be marked as "Official" to facilitate the distinction between configurations issued by the manufacturer and self-signed configurations
- **Status**: Whether the configuration package has been applied, whether it is valid, etc.

Comparison between Secure Storage and File Key

Helps you understand why TianshanOS uses secure storage (NVS) by default instead of files
.

Comparison Table

Feature	Non-Volatile Storage (NVS)	File Key (SD Card)
Private Key Protection	Hardware NVS encrypted partition, difficult to extract	Plaintext PEM files are easily copied and leaked
Convenience of use	Simply remember the key ID, no need to manage file paths	Needs to remember the file path, which is easy to confuse
Key Backup	You must check "Exportable" to back up	Files can be directly copied for backup
Cross-device usage	Key binds to a single device	

		Can be copied to multiple devices for use
Prevent accidental deletion	Requires explicit deletion operation	File system errors may cause loss
Storage Capacity	NVS is limited (approximately 8-32 keys)	SD card has large capacity
Access speed	Fast (NVS Read)	Slow (SD Card I/O)
Security Score	High (5 stars)	Low (2 stars)
Recommended Scenario	**Production environment, long-term use, security-sensitive**	Temporary Use, Key Migration, Multi-Device Sharing

Advantages of Secure Storage

1. **Hardware-level protection**:
   - NVS partition supports Flash encryption (can be enabled in production environment)
   - Private keys are difficult to extract physically

2. **Default Security**:
   - Private keys are not exportable by default (unless explicitly checked)
   - Prevent leakage caused by misoperation

3. **Centralized Management**:
   - All keys are uniformly viewed and managed in the WebUI
   - will not be scattered throughout the file system

4. **Permission Control**:
   - Can set "Hide Key" to prevent low-privilege users from accessing
   - Cooperate with the "Exportable" tag to achieve fine-grained control

When to Use File Keys

Although secure storage is more secure, the following scenarios can use file keys:

- **Key Migration** : Import existing keys from other devices
- **Temporary Test** : For temporary use, delete after use
- **Multi-device sharing**: The same key needs to be used on multiple devices
- **External Tools**: Need to be used in a desktop SSH Client (such as PuTTY, Terminal)

CLI command using file key:

代码块

```
# 使用 SD 卡上的密钥文件连接
ssh --host 192.168.1.100 --user nvidia --key /sdcard/id_rsa --shell
```

Frequently Asked Questions and Troubleshooting

7.1 Key Management Issues

Q1: When generating a key, it prompts "Storage space is full"?

Phenomenon : After clicking "Generate", an error "NVS storage full" or "Key count has reached the limit" is displayed.

Reason: The NVS partition has limited capacity (48KB), with a maximum storage of:

- Approximately 8 RSA-4096 keys
- Approximately 12 RSA-2048 keys

- Approximately 32 ECDSA P-256 keys

Solution :

1. Enter the "Key Management" list to view existing keys
2. Delete keys that are no longer needed (revoke deployed hosts first)
3. Delete and then regenerate a new key

Suggestion :

- Regularly clean up expired keys
- Use ECDSA P-256 (small footprint) preferentially
- Avoid generating too many RSA-4096 keys

Q2: ECDSA key fails to connect to SSH server?

Symptom: Failure occurs when deploying or connecting using ECDSA type keys, with error messages "Authentication failed" or "Unsupported key type".

Reason: The SSH Client of the current version of TianshanOS does not currently support public key authentication for ECDSA keys (known limitation).

Solution :

1. Delete this ECDSA key
2. Select **RSA 2048-bit** or **RSA 4096-bit when regenerating keys**
3. Redeploy Public Key

Instructions :

- The ECDSA key itself is secure (it is fine for use in HTTPS certificates)
- ECDSA support for SSH public key authentication will be added in future versions

Q3: How to back up the key?

Scenario: Concerned about device damage or key loss during firmware upgrade, backup is required.

Method 1: Export Private Key (Requires Prior Setup)

1. Check **"Allow Private Key Export" when generating the key**
2. After generation, click the "Private Key" button
3. Download as `.pem` file
4. Stored in secure offline media (encrypted USB drive, password manager)

Method 2: Record the Key ID and Configuration

1. Record key ID, type, and remarks
2. Record which servers the deployment is made to
3. If the key is lost, regenerate and redeploy

Recommendation:

- Important production environment keys: Check "Exportable" and back up
- Test environment key: Not backed up, regenerated after loss

---

Q4: Can I recover the key after deleting it?

Answer: **Unable to recover**.

Delete operations are permanent:

- Key has been completely cleared from NVS

- No recycle bin or undo function
- The only way to recover: previously exported private key backup

Checklist before deletion:

1. Confirm that this key has not been deployed to critical servers
2. If there is a deployment, it has been revoked or removed
3. Private key has been exported if backup is needed
4. has notified other personnel who may use this key

Remedy after accidental deletion:

- If there is a backup: Re-import via the `key --import` command in the CLI
- If there is no backup: regenerate a new key and redeploy it to all servers

---

7.2 Deployment and Connection Issues

Q5: When deploying the public key, it prompts "Connection timed out"?

Phenomenon : After clicking "Start Deployment", waiting for a long time, and finally displaying "Connection timeout".

Possible Reasons :

1. Network is unreachable (device cannot access the target host)
2. The target host has not enabled the SSH service
3. Firewall blocks port 22
4. Incorrect IP address or port entered

Troubleshooting Steps :

Step 1: Check network connectivity

- Ping the target host in the terminal (if you have root privileges)
- Or ping the host on another device to confirm that the host is online

Step 2: Check the SSH service

- Run on the target server: `systemctl status sshd` (or `service ssh status`)
- Confirm that the SSH service is running

Step 3: Check the firewall

- Device Firewall: Confirm that outbound SSH connections are allowed
- Server Firewall: Confirm that inbound port 22 (or custom port) is allowed

代码块

```
# Check the status of the firewall (server side)
sudo ufw status
sudo firewall-cmd --list-all
```

Step 4: Verify Parameters

- Confirm that the IP address, port, and username are filled in correctly
- Try to SSH connect from other devices to verify server accessibility

---

Q6: Why does it prompt "Incorrect password" during deployment?

Symptom: After entering the password, it prompts "Authentication failed" or "Incorrect password".

Possible Reasons :

1. Password input error (case sensitivity, special characters)
2. Username is incorrect

3. SSH Configuration Disables Password Authentication
4. User account is locked

Solution :

1. **Reconfirm Password** :
   - Pay attention to case sensitivity (Linux passwords are case-sensitive)
   - Pay attention to special characters (spaces, quotes, etc.)
   - Test if the password is correct elsewhere

2. **Check Username** :
   - Confirm that the username is spelled correctly
   - Password login may be disabled for the root user on some systems

3. **Check SSH configuration** (server side):

   代码块

   ```
   # Check SSH configuration
   sudo cat /etc/ssh/sshd_config | grep PasswordAuthentication

   # Make sure the following configurations are set to yes
   PasswordAuthentication yes

   # Restart the SSH service after the modification
   sudo systemctl restart sshd
   ```

4. **Check user status** :

   代码块

   ```
   # Check if the user is locked out
   sudo passwd -S <username>
   ```

Q7: Deployment was successful, but a password is still required?

Phenomenon : Deployment shows success, but still prompts for a password when connecting with a key.

Possible Reasons :

1. `authorized_keys` file permissions are incorrect
2. SSH configuration does not enable public key authentication
3. The permissions of the main directory are too loose
4. SELinux Context Error (RHEL/CentOS)

Solution :

1. **Check file permissions** (executed on the server side):

   代码块

   ```
   # Check permissions
   ls -ld ~/.ssh
   ls -l ~/.ssh/authorized_keys

   # The correct permissions should be:
   # drwx------ (700) ~/.ssh
   # -rw------- (600) ~/.ssh/authorized_keys

   # If the permissions are incorrect, fix:
   chmod 700 ~/.ssh
   chmod 600 ~/.ssh/authorized_keys
   ```

2. **Check the file content** :

   代码块

   ```
   # Check the content of authorized_keys
   cat ~/.ssh/authorized_keys

   ```

```
# should include lines that start with ssh-rsa or ecdsa-sha2-
```

3. **Check SSH configuration** (server side):

代码块

```
sudo cat /etc/ssh/sshd_config | grep -E "PubkeyAuthentication|AuthorizedKeysFile"

# Ensure the following configuration
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
```

4. **Check SELinux** (RHEL/CentOS):

代码块

```
# Restore SELinux context
restorecon -R ~/.ssh
```

5. **Restart the SSH service** (if the configuration has been modified):

代码块

```
sudo systemctl restart sshd
```

Q8: Why is the "Deployed Hosts" list empty?

Phenomenon : The public key has been deployed, but the list shows "No deployed hosts yet".

Reason: Only hosts deployed via the WebUI "Deployment" feature will be automatically added to this list.

Situations where it does not appear in the list :

- Deployed via the terminal CLI's `ssh --copyid`
- Manually copy the public key to the `authorized_keys of the server`
- Public keys deployed by other devices or tools

Solution :

- If it is indeed necessary to manage in the WebUI, you can redeploy once via the "Deployment" function of the WebUI; or use the "Import Host" to import the.tscfg configuration package containing the host information to make the list consistent with the actual situation
- or view and manage solely through the `hosts --deployed` command in the CLI

Design Description:

- List records are stored locally on the device (`ts_ssh_hosts_config`)
- Only records hosts deployed via WebUI or specific APIs
- The purpose is to provide a unified deployment management interface

---

Q8b: When importing a host or an import configuration package, it prompts "Signature verification failed"?

Symptom: After clicking "Import Host" or "Import Configuration Package" and selecting/pasting a.tscfg file, a prompt indicates that signature verification failed or decryption is not possible.

Possible Reasons :

- .tscfg file is corrupted or copied incompletely (missing head or tail, truncated)
- The configuration package was not signed with a certificate trusted by the current device (or target device)

- The device system time deviation is too large, resulting in the failure of certificate validity period verification
- The pasted JSON has a formatting error (extra spaces, missing quotes, etc.)

Solution :

1. **Check file integrity**: Reobtain.tscfg from the source to ensure it contains complete `-----BEGIN...-----` / `-----END...-----` blocks, or re-export
2. **Confirm the issuance source** : Confirm that the configuration package is issued by the local machine certificate or the official/target device certificate you trust; when importing a host, if it was exported as "self-encrypted", it must be imported on the same device or a device with the same certificate
3. **Verify device time**: Check if the device date and time are correct on the System/Network page, and synchronize NTP if necessary
4. **Verify Only**: In the configuration package import pop-up window, first click <Verify Only>, view the specific error message (such as signer, certificate CN), and then troubleshoot according to the prompt

---

7.3 Known Host Fingerprint Issues

Q9: Is the "Known Host Fingerprints" list empty?

Phenomenon : When opening the security page, the list shows "No known hosts", but an SSH server has already been connected to.

Possible Reasons :

Reason 1: It has never been connected

- Solution: Perform an SSH connection (deploy the public key or use key-based connection), and after trusting the host, the list will have a record

Reason 2: Module not initialized (known bug)

- Phenomenon: The host has been connected and trusted, but the list remains empty; it suddenly appears after refreshing the page
- Reason: `The ts_known_hosts module uses lazy initialization and is not actively initialized when the security service starts`
- Solution:
  a. Perform an SSH connection (trigger initialization)
  b. Refresh the security page
  c. The list should display records

Reason 3: NVS data corruption

- Phenomenon: Terminal CLI `hosts --list` is also empty
- Solution: Reconnect and trust the host, then rebuild the records

How to confirm if there are fingerprint records (requires root privileges to access the terminal):

代码块

```
# View list of known hosts
hosts --list

# View details
hosts --list --json
```

Q10: How to handle the host fingerprint mismatch warning?

Symptom: A red warning "Security Warning: Host fingerprint does not match" pops up during connection.

Correct handling process (see details in Section 4.4):

1. **Stop Immediately**: Do not blindly click "Update Host Key"
2. **Contact the administrator**: Confirm whether the server has been reinstalled or the key has been changed
3. **After confirming legality** : Click "Update Host Key"
4. **When unable to confirm** : Click "Cancel" to troubleshoot network security

Wrong Example:

- Click "Update Host Key" when you see a warning (you may have been attacked by a man-in-the-middle)
- Close the warning and continue connecting (the system will block it)
- Ignoring warnings and assuming them to be system bugs (this is an important security feature)

Safety Notice :

- Host fingerprint mismatch is**a serious security incident**
- Must be verified before proceeding
- If it occurs frequently, the network environment may be insecure

---

Q11: How to verify if the host fingerprint is correct?

Scenario: When connecting for the first time or when the fingerprint does not match, it is necessary to verify the authenticity of the server fingerprint.

Method 1: Obtain the official fingerprint on the server side

1. **Log in to the server** (via Console or other trusted means)

2. **View the host key fingerprint**:

```
代码块
# ECDSA key (recommended)
ssh-keygen -lf /etc/ssh/ssh_host_ecdsa_key.pub

# ED25519 Key (More secure, newer)
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub

# RSA keys (good compatibility)
ssh-keygen -lf /etc/ssh/ssh_host_rsa_key.pub
```

3. **Output Example**:

```
代码块
256 SHA256:xYzAbC123456789aBcDeF/gHiJkLmNoPqRsTuVwXyZ01= root@server (ECDSA)
```

4. **Compare Fingerprint**:
   - Compare `SHA256:xYz...` with the fingerprint displayed on the device
   - Must**be exactly the same**(character-by-character comparison)

Method 2: Contact the server administrator

1. Contact the administrator responsible for this server
2. Request to provide the official host key fingerprint (SHA256 format)
3. Comparison Confirmation

Method 3: Use other trusted devices for verification

1. Use other computers or devices that have been confirmed to be secure
2. SSH into the server and record the fingerprint
3. Compare whether the fingerprint matches the one displayed by TianshanOS

Safety Notice :

- For high-security scenarios (production environment, finance, healthcare), it is recommended to verify the fingerprint before the first connection
- The internal test environment can appropriately simplify the process

---

7.4 HTTPS Certificate Issues

Q12: Why is the Generate CSR button grayed out and unavailable?

Symptom: The "Generate CSR" button is grayed out and cannot be clicked.

Reason: The key pair has not been generated yet. CSR needs to be signed with a private key, so a key pair must be generated first.

Solution :

1. Click **the "Generate Key Pair"** button
2. Waiting for successful key generation
3. The "Generate CSR" button will automatically become available (blue)

Related Buttons:

- "Install Certificate" and "Install CA" also require generating a key pair first
- Only "Generate Key Pair" and "Delete Credential" are available in any state

---

Q13: When installing the certificate, it prompts "The certificate does not match the private key"?

Phenomenon : After pasting the certificate content and clicking "Install", an error "Certificate and private key do not match" is displayed.

Reason:

- This certificate was not issued using the CSR generated by the **current device key** .
- May be:
  - Used CSR from other devices
  - Regenerated the key pair after generating the CSR (resulting in a change to the private key)
  - An incorrect CSR was used when issuing the CA

Solution :

Method 1: Regenerate CSR and Sign

1. Click "Generate CSR" again on the device
2. Copy the new CSR content
3. Submit to CA for reissuance
4. Install the newly issued certificate

Method 2: Delete the voucher and start over

1. Click "Delete Voucher"
2. Restart the full process from "5.3.1 Generate Key Pair"
3. Ensure that the same key pair is used for CSR, issuance, and installation

Preventive Measures:

- Do not regenerate the key pair after generating the CSR
- Confirm the correct source of the CSR before issuance
- Mark the file names (e.g. `device-001.csr` , `device-001.crt`)

Q14: Why does the certificate status show "Expiring Soon"?

Phenomenon : The status card shows "About to expire", and the remaining days < 30.

Handling Suggestion:

- **30 - 15 days**: Start preparing for renewal, not urgent
- **15-7 days**: Renew as soon as possible to avoid affecting services
- **< 7 days** : Renew immediately, high priority

Renewal Process (Keep Key):

1. **Generate New CSR**:
   - Click "Generate CSR"
   - can use the same CN (Device ID)
   - Copy CSR Content

2. **Submit for CA Issuance**:
   - Submit the CSR to your CA
   - Indicates a renewal operation (which may expedite the approval process)

3. **Install New Certificate** :
   - After obtaining the newly issued certificate
   - Click "Install Certificate"
   - Paste the content of the new certificate
   - The old certificate is automatically overwritten

4. **Verify Renewal**:
   - Check that the "Expiration Date" has been updated to the new date
   - Check that "Remaining Days" has been reset

Precautions :

- The old certificate remains valid during the renewal period (until it expires)
- The new certificate takes effect immediately after installation

- CA chains usually do not need to be updated (unless the CA itself expires)
- If the CA chain has also expired, it is necessary to update the CA chain simultaneously

---

Q15: Will the HTTPS service still work after deleting the certificate?

Answer: Depending on the system configuration, the following situations may occur.

Scenario 1: Downgrade to HTTP

- HTTPS service is disabled, and WebUI is accessed via HTTP (port 80)
- Security is reduced, and data is transmitted in plaintext

Scenario 2: Use a self-signed certificate

- The system automatically generates a temporary self-signed certificate
- The browser will display a "Certificate is not trusted" warning
- Requires manual trust to access

Case 3: Service Unavailable

- If the configuration requires the use of mTLS, the HTTPS service may reject the connection
- Reinstalling the certificate is required to resume

Suggestion :

- Before deleting vouchers, confirm system configuration and backup plans first
- The test environment can be safely deleted
- It is recommended to install a new certificate before deleting it in the production environment to avoid service interruption

---

7.5 Permission and Authentication Issues

Q16: Can the admin user use all the functions of the security page?

Answer: **Yes**.

Admin Permission Scope:

- All functions of the security page
- System, Network, File Page
- Terminal, automation, and instruction pages require root privileges and cannot be accessed by admin

Security Page Function List (Available to admin):

- Generate, delete, and export SSH keys
- Deploy the public key to the remote server
- Revoke the deployed public key
- Manage deployed host records (including importing host configurations)
- View and delete known host fingerprints
- Generate, install, and delete HTTPS certificates
- Generate CSR, Install CA Chain
- Configuration Package: Export Device Certificate, Import Configuration Package, View Configuration Package List

If root permission functionality is required :

- Please contact the system administrator and log in using the root account
- or apply for an upgrade to root privileges

Q17: When exporting the private key, it prompts "Insufficient permissions" or the button is grayed out?

Phenomenon : When clicking the "Private Key" button:

- The button is displayed in gray (disabled) and cannot be clicked
- or click to prompt "This key cannot export the private key"

Reason: When this key was generated**, the option** "Allow private key export" was not checked.

TianshanOS Security Design :

- By default, the private key **is non - exportable** (security best practice)
- Only keys with the "Allow Exporting Private Key" option explicitly checked can export their private keys
- This limitation is not related to user permissions (admin/root), but rather an attribute of the key itself

Solution :

Method 1: Generate a new exportable key

1. Check when generating a new key **"Allow private key export"**
2. After generation, the private key can be exported for backup
3. Deploy to the required server

Method 2: Export only the public key

- If you don't need the private key, you can export the public key (all keys can export the public key)
- Public keys can be securely shared without the risk of leakage

Method 3: Accept the Limitations

- If the key has been deployed to the production server, keeping it in a non-exportable state is actually more secure

- Managed through the WebUI "Deployment" feature, no need to export the private key

Design Concept:

- Private keys are the most sensitive security credentials and should be avoided being exported as much as possible
- Export only when backup or migration is truly necessary
- By default, it cannot be exported, reducing the risk of accidental operation and leakage

---

Best Practice Recommendations

8.1 Key Management Best Practices

1. Key Naming Convention

Recommended Naming Convention:

- `<Usage>_<Environment>` : e.g. `agx_prod` , `lpmu_test`
- `<Server>_<User>` : Such as `server01_root` , `gateway_admin`
- `<Project>_<Serial Number>` : e.g. `project_a_01` , `iot_device_001`

Avoided naming :

- `key1` , `test` , `temp` (meaningless)
- Chinese or special characters (which may cause compatibility issues)
- An overly long name (inconvenient for terminal input)

2. Key Type Selection

Recommendation Strategy:

- **Daily Use**: RSA 2048-bit (Good Compatibility, Fast Generation)
- **High-security scenario**: RSA 4096-bit (military-grade encryption, slow generation)
- **ECDSA**: Currently only used for HTTPS certificates, not for SSH

Capacity Planning:

- RSA 2048: Approximately 12 can be stored
- RSA 4096: Can store approximately 8
- Hybrid Use: Select Based on Server Importance

3. Private Key Export Control

Recommendation Strategy:

- **Production environment key**: Do not check "Allow export", and it will be permanently stored on the device
- **Backup Key**: Check "Allow Export" and regularly backup to offline media
- **Test Key**: Not checked, regenerate after loss

Security Checklist for Exporting Private Keys:

- [ ] Confirm that the export is necessary (backup, migration)
- [ ] Use encrypted transmission methods (encrypted USB disk, VPN)
- [ ] Store in a secure location (password manager, encrypted disk)
- [ ] Delete temporary files immediately after use
- [ ] Regularly review who has a copy of the private key

4. Regularly review keys

Recommended Frequency:

- **Quarterly**: Check the key list and remove keys that are no longer in use
- **Every six months** : Review deployed hosts and revoke unnecessary access
- **Annually** : Consider rotating the keys of critical servers

Review Inspection Items:

- [ ] Which keys have not been used for a long time?
- [ ] Which keys are deployed to the deactivated servers?
- [ ] Is there a test key that was forgotten to be deleted?
- [ ] Is the storage space approaching its limit?

8.2 Deployment Management Best Practices

1. Pre-deployment Record

It is recommended to use spreadsheets or documents for recording :

Key ID	Deploy to Server	Username	Usage	Deployment Date	Responsible Person
agx_prod	192.168.1.100	nvidia	AGX Monitoring	2026-01-30	Zhang San
lpmu_test	10.10.99.50	root	LPMU Test	2026-01-28	Li Si

Benefits:

- Facilitates subsequent maintenance and privilege auditing
- Quick handover during personnel changes
- Quickly locate during troubleshooting

2. Test immediately after deployment

Process:

1. After successful deployment, click the "Test" button
2. Confirm connection successful
3. If it fails, troubleshoot immediately and do not leave it for later

Test content:

- Is the connection successful?
- Is the username correct?
- Is it possible to execute the command

3. Promptly revoke unnecessary access

Scene:

- Project completed, no longer need to access
- Personnel departure, revoke personal key
- Server offline or migrated

Operation:

1. Click "Revoke" in the "Deployed Hosts" list
2. Or manually delete the corresponding line in `authorized_keys` on the server side

4. Key Rotation Plan

Recommendation Strategy:

- **Key Server** : Rotate keys once a year
- **General Server** : Rotate every 2-3 years or as needed
- **Test Environment**: Rotate after project completion

Rotation Process:

1. Generate a new key (new ID, e.g., `agx_2027`)

2. Deploy new keys to all servers
3. Test new key connection
4. Revoke the old key
5. Delete the old key

8.3 Best Practices for Host Fingerprint Management

1. First connection verification

Standard Process (High-Security Environment):

1. Obtain the official fingerprint from the administrator before connecting
2. Compare the fingerprint displayed by the system when connecting
3. After confirming consistency, enter `yes` trust
4. Record the trust time and verification personnel

Simplified Process (Internal Test Environment):

1. Confirm that the IP address is correct
2. Direct input `yes` trust
3. Subsequent reliance on the automatic verification mechanism

2. Principles for Handling Fingerprint Mismatch

Ironclad Rule: **Never ignore fingerprint mismatch warnings**

Processing Priority:

1. **First Priority**: Verify whether the server has been reinstalled/replace the key
2. **Second Priority**: Check the network environment (whether it is on an insecure network)
3. **After confirmation** : Only then update the host key

Record Change:

- Record who updated the fingerprint, the reason, and the time
- Retain evidence of server reinstallation or key replacement

3. Regularly clean up expired records

Cleanup Object:

- Deactivated Server
- Migrated IP Address
- Temporary Server for Test Environment

Operation:

- Review the list of "Known Host Fingerprints" quarterly
- Delete host records that are no longer in use
- Keep the list tidy for easy management

8.4 Certificate Management Best Practices

1. Early Renewal

Renewal Schedule:

- **Certificate validity period > 30 days** : No action required, regular inspection
- **30 - 15 days**: Prepare the renewal process, contact the CA, or prepare documents
- **15 - 7 days**: Execute renewal, obtain a new certificate, and install it
- **<7 days**: Urgent renewal, high priority

Automatic Reminder (Suggestion):

- Set a calendar reminder (30 days before certificate expiration)
- or use the monitoring system to check the certificate status

2. Back up the CA certificate chain

Why Back Up:

- CA chain is the same on all devices (same PKI system)
- No need to re-acquire when configuring other devices after backup
- Can still be used when the CA server fails

Backup Method:

1. Click "View Certificate" to obtain the device certificate
2. Save the CA chain separately as `ca-chain.pem`
3. Stored in a secure document library or Configuration Management System

Use Backup:

- Use directly when configuring a new device
- Emergency use when CA server is unavailable

3. Test Environment First

Certificate Update Process (Multi-Device Environment):

1. First execute the complete process on the test device
2. Verify that the new certificate works properly
3. Verify that the mTLS connection is successful
4. and then extended to production equipment

Test Item:

- [ ] Certificate installation successful
- [ ] HTTPS service started successfully

- [ ] mTLS Mutual Authentication Successful
- [ ] Client can access

4. Regularly check the certificate status

Check Frequency:

- **Monthly**: Open the security page and view the status card
- **Quarterly**: Conduct a detailed review of certificate information to confirm no abnormalities
- **After a major change**: The verification certificate remains valid

Check Item:

- [ ] Is the status "Activated"?
- [ ] Is the validity period sufficient (> 30 days)?
- [ ] Is CN correct?
- [ ] Is the issuer the expected CA?

Reference Document Link

The following documents provide more in-depth technical details and troubleshooting, and can be accessed via in-app links after deployment to Mintlify.

Dig into technical details

- **Security Implementation**:
  - Security Level Definitions (L1-L4)
  - Key Storage Principles (NVS Encryption, Capacity Planning)
  - SSH Authentication Mechanisms (Password, Public Key, Secure Storage)

    - WebUI Authentication System (User System, Permission Levels, Session Management)

- **PKI Design**:
    - mTLS Architecture Design
    - CA Hierarchy (Root CA, Intermediate CA)
    - Certificate Type and Policy
    - Device Activation Process
    - Certificate Update Mechanism

Command Line Operations

- **Command Reference**:
    - `key` command: All CLI operations for key management
    - `ssh` Command: Complete parameters for SSH connection, deployment, and revocation
    - `hosts` command: CLI management of known hosts and deployed hosts
    - `cert` command (if any): CLI operations for certificate management

Troubleshooting

- **Troubleshooting** :
    - Debugging SSH Connection Issues
    - Solution for the known issue where the host list is initially empty
    - Troubleshooting Steps for Certificate Verification Failure
    - Diagnosis of Network Connectivity Issues

Quick Start

- **Quick Start Guide**:
    - Overall Introduction to TianshanOS

- WebUI Page Navigation
- Brief Introduction to the Security Page
- Initial Configuration Wizard

Conclusion

This document covers all the functions and operating methods of the TianshanOS security page. As **an admin user** , you can use these functions to manage SSH keys, remote host connections, and HTTPS certificates.

We recommend that you :

1. Read this document in full before first use
2. Practice various operations in the test environment
3. Bookmark this document for easy reference at any time
4. Consult relevant technical documents for in-depth study as needed

Safety Reminder :

- Properly store keys and certificates
- Pay attention to all security warnings (such as host fingerprint mismatch)
- Regularly review and update keys and certificates
- Record important operations and changes

If you have any questions or suggestions, please contact the system administrator or refer to the project document library.